

SOFTWARE REQUIREMENTS SPECIFICATION

(Prepared in accordance with IEEE 29148 principles)

Context-Aware Dynamic Token Mapper

An AI-Assisted Adaptive Authorization Extension for Keycloak

Field	Value
Document Type	Software Requirements Specification (SRS)
Companion Document	Internship Project Concept Note — Context-Aware Dynamic Token Mapper
Target Platform	Keycloak Open-Source IAM Server
Domain	Identity and Access Management (IAM), Adaptive / Risk-Based Authorization
Standard Followed	IEEE 29148:2018 (Systems and software engineering — Requirements engineering)
Status	Draft — For Review
Author	Bereket Kiros — Mekelle University
Version	1.0

This Software Requirements Specification (SRS) expands the accompanying Concept Note into a complete, implementation-ready requirements baseline. It does not redefine the project's architecture, objectives, or scope; it elaborates each architectural decision already established in the Concept Note into verifiable, traceable requirements suitable for design, implementation, and independent verification.

Table Contents

Table of Contents.....	2
1 Introduction.....	6
1.1 Purpose.....	6
1.2 Scope.....	6
1.2.1 In Scope	6
1.2.2 Out of Scope	7
1.3 Intended Audience	7
1.4 Definitions.....	7
1.5 Acronyms.....	8
1.6 References.....	8
1.7 Document Overview	9
2 Overall Description.....	10
2.1 Product Perspective.....	10
2.2 Product Functions	10
2.3 User Classes and Characteristics.....	11
2.4 Operating Environment.....	11
2.5 Design Constraints	11
2.6 Assumptions.....	12
2.7 Dependencies	12
3 System Features	13
3.1 Contextual Authentication Data Collection	13
3.2 Context Validation	13
3.3 AI Risk Assessment Service Communication	14
3.4 Risk Evaluation (AI Risk Assessment Service).....	14
3.5 Risk-Based Token Claim Transformation	14
3.6 Resilience and Fallback Handling.....	15
3.7 Configuration Management	15
3.8 Logging and Observability	16
4 Functional Requirements	17
4.1 Authentication Context Collection (FR-001 – FR-005).....	17
4.2 Context Validation (FR-006 – FR-009).....	19
4.3 REST Communication (FR-010 – FR-014).....	20

4.4 Risk Evaluation (FR-015 – FR-018).....	22
4.5 Token Claim Modification (FR-019 – FR-021).....	23
4.6 Scope Suppression (FR-022 – FR-024)	24
4.7 Role Suppression (FR-025 – FR-026)	25
4.8 Risk Claim Insertion (FR-027 – FR-028)	26
4.9 Restricted Claim (FR-029 – FR-030).....	26
4.10 Logging (FR-031 – FR-034)	27
4.11 Configuration (FR-035 – FR-038)	28
4.12 Timeout Handling (FR-039 – FR-040)	30
4.13 Retry Logic (FR-041 – FR-042)	30
4.14 Fallback Behaviour (FR-043 – FR-045)	31
4.15 Health Checking (FR-046 – FR-047).....	32
4.16 Deployment (FR-048 – FR-049).....	33
4.17 Administration (FR-050 – FR-051)	33
5 Non-Functional Requirements	35
5.1 Performance	35
5.2 Reliability.....	35
5.3 Availability	35
5.4 Maintainability.....	35
5.5 Scalability	35
5.6 Security	36
5.7 Extensibility	36
5.8 Configurability.....	36
5.9 Observability.....	36
5.10 Testability	36
5.11 Portability.....	37
5.12 Interoperability.....	37
5.13 Usability.....	37
5.14 Compatibility	37
6 External Interface Requirements.....	38
6.1 User Interfaces	38
6.2 Administrative Interfaces.....	38
6.3 Hardware Interfaces	38
6.4 REST API and JSON Contracts.....	38
6.4.1 Risk Scoring Endpoint	38

6.4.2	Request Payload Fields	39
6.4.3	Response Payload Fields.....	39
6.4.4	Health Check Endpoint	40
6.4.5	Status Codes and Error Responses.....	40
6.4.6	Required Headers	40
7	Security Requirements	41
7.1	JWT Security	41
7.2	REST Security	41
7.3	Input Validation	41
7.4	Output Validation.....	41
7.5	Logging Security.....	42
7.6	Least Privilege	42
7.7	Zero Trust Principles.....	42
7.8	Secure Configuration	42
7.9	Timeout Handling (Security Perspective).....	42
7.10	Failure Behaviour (Security Perspective)	42
8	System Models.....	43
8.1	Use Case Diagram.....	43
8.2	Component Diagram.....	44
8.3	Deployment Diagram.....	45
8.4	Class Diagram.....	45
8.5	Package Diagram	46
		46
8.6	Activity Diagram	47
		47
8.7	Sequence Diagram	48
		48
8.8	Data Flow Diagram.....	49
		49
8.9	State Diagram.....	50
		50
8.10	Network Diagram.....	51
9	Use Cases	52
9.1	UC-01: Normal (Low-Risk) Login	52
9.2	UC-02: High-Risk Login	53

9.3 UC-03: AI Risk Assessment Service Failure.....	54
9.4 UC-04: Timeout Scenario	54
9.5 UC-05: Token Generation (Claim Transformation Detail).....	55
9.6 UC-06: Mapper Configuration by an Administrator.....	56
9.7 UC-07: System Startup	57
9.8 UC-08: System Shutdown.....	58
10 Configuration Requirements	59
11 Deployment Requirements.....	62
11.1 Development Environment	62
11.2 Operating Systems	62
11.3 Build Process	62
11.4 Packaging.....	62
11.5 Deployment Steps	63
12 Testing Requirements	64
12.1 Testing Categories	64
12.2 Test Cases	64
13 Risks.....	69
13.1 Technical Risks.....	69
13.2 Operational Risks.....	69
13.3 Security Risks	70
13.4 Performance Risks	70
13.5 Integration Risks	70
14 Future Enhancements.....	71
14.1 Version 2 — Machine Learning Risk Engine.....	71
14.2 Version 3 — Advanced AI.....	71
14.3 Version 4 — Potential Reinforcement Learning-Based Adaptive Policy Engine.....	71
14.4 Architectural Stability Across Versions.....	72
15 Appendices.....	73
15.1 Glossary	73
15.2 Abbreviations.....	73
15.3 Technical References	74
15.4 Standards and Concepts Referenced.....	74

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional, non-functional, interface, and security requirements for the Context-Aware Dynamic Token Mapper, an AI-assisted adaptive authorization extension for the Keycloak Identity and Access Management (IAM) platform. The purpose of this document is to translate the architectural decisions and objectives already established in the project's Concept Note into a detailed, verifiable, and implementation-ready requirements baseline that can be used by developers, reviewers, and testers throughout the internship project lifecycle.

This document exists to remove ambiguity between what the Concept Note describes at a conceptual level and what must actually be built, configured, and tested. It does not introduce new architectural decisions, alter component responsibilities, or change the scope defined in the Concept Note; instead, it expands each concept into concrete, testable requirements, each carrying a unique identifier, rationale, and acceptance criteria.

1.2 Scope

The system specified in this document is a custom OpenID Connect (OIDC) Protocol Mapper for Keycloak, together with a companion external AI Risk Assessment Service, that together implement session-scoped, context-aware modification of JWT token claims at token-issuance time. The Protocol Mapper is packaged and deployed as a Keycloak Service Provider Interface (SPI) provider; the AI Risk Assessment Service is an independently deployable REST service.

As established in the Concept Note, the scope of Version 1 of the system is restricted to a rule-based risk engine used to validate the architecture and the Keycloak integration pattern. Machine learning and advanced AI capabilities are explicitly out of scope for Version 1 and are documented in Section 14 (Future Enhancements) as a planned roadmap rather than as requirements of the current version.

1.2.1 In Scope

- A custom Java class extending AbstractOIDCProtocolMapper, packaged as a deployable Keycloak provider JAR.
- Collection of contextual authentication signals available from the Keycloak authentication/user session (IP address, timestamp, user-agent/device characteristics, accessible login history).
- A minimal, purpose-built AI Risk Assessment Service (Python/FastAPI) exposing a REST scoring endpoint backed by a rule-based scoring function.
- Synchronous REST/HTTP/JSON integration between the mapper and the AI Risk Assessment Service, including timeout, retry, and fallback handling.
- Session-scoped, in-memory token claim transformation: scope/role suppression, risk_level claim insertion, and restricted claim insertion.
- Configuration of the mapper through the Keycloak Admin Console (endpoint, timeout, retry count, risk threshold, claims to add/remove, logging level).
- Functional, integration, and security testing in a local/self-hosted, Docker-based Keycloak development environment.

1.2.2 Out of Scope

- Modification of Keycloak's core authentication engine, Admin Console, or database schema.
- Production-grade device fingerprinting or behavioral-biometrics subsystems.
- Research, training, or tuning of a sophisticated machine learning risk model, or the collection of large-scale labeled datasets.
- Permanent alteration of user roles, groups, or attributes stored in the Keycloak database.
- Multi-tenant, high-availability, or large-scale production deployment and load testing.
- Comprehensive security hardening, formal threat modeling, or compliance certification of the AI Risk Assessment Service.
- Reinforcement-learning-based adaptive policy engines (documented only as a Version 4 future enhancement).

1.3 Intended Audience

Audience	Interest in this Document
Project Developer / Internship Candidate	Primary implementer; uses this SRS as the authoritative specification for design and coding.
Academic / Internship Supervisor	Reviews the requirements baseline for completeness, feasibility, and alignment with the Concept Note.
Software Architecture Reviewers	Evaluate architectural conformance, separation of concerns, and extensibility of the design.
Quality Assurance / Test Engineers	Derive test plans, test cases, and acceptance criteria directly from Sections 3, 4, 5, and 12.
Future Open-Source Contributors	Use the SRS to understand system behavior and configuration before extending or reusing the mapper.
Security Reviewers	Assess the security posture of the mapper-to-AI integration and token claim handling described in Section 7.

1.4 Definitions

Term	Definition
Protocol Mapper	A Keycloak extension point that transforms user, client, or session attributes into claims of an issued OIDC token.
Claim	A piece of information asserted about a subject within a JWT, expressed as a name/value pair.
Risk Classification	A categorical output (e.g., LOW, MEDIUM, HIGH) produced by the AI Risk Assessment Service representing the assessed risk of an authentication event.
Contextual Signal	Any authentication-time attribute (e.g., IP address, timestamp, user-agent) collected and forwarded to the AI Risk Assessment Service.
Session-Scoped Modification	A change applied only to the in-memory token under construction for the current authentication session, with no effect on persisted data.
Fallback Behaviour	The deterministic, predefined claim state applied when the AI Risk Assessment Service is unreachable, times out, or returns an invalid response.

Term	Definition
Restricted Claim	A boolean or status claim inserted into the token indicating that the session is operating under reduced privileges.

1.5 Acronyms

Acronym	Expansion
IAM	Identity and Access Management
SPI	Service Provider Interface
OIDC	OpenID Connect
JWT	JSON Web Token
SSO	Single Sign-On
REST	Representational State Transfer
API	Application Programming Interface
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
TLS	Transport Layer Security
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
ML	Machine Learning
CI/CD	Continuous Integration / Continuous Deployment

1.6 References

- IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.
- Internship Project Concept Note — Context-Aware Dynamic Token Mapper: An AI-Assisted Adaptive Authorization Extension for Keycloak (companion document to this SRS).
- Keycloak Server Developer Guide — Service Provider Interfaces and Protocol Mappers.
- OAuth 2.0 Authorization Framework (RFC 6749).
- OpenID Connect Core 1.0 specification.
- JSON Web Token (JWT) standard (RFC 7519).
- NIST SP 800-207 — Zero Trust Architecture.

1.7 Document Overview

The remainder of this document is organized as follows. Section 2 describes the overall product context, functions, user classes, and constraints. Section 3 presents system features at a functional-area level. Section 4 enumerates detailed, uniquely identified functional requirements. Section 5 specifies non-functional requirements. Section 6 defines external interface requirements, including the REST API contract between the mapper and the AI Risk Assessment Service. Section 7 specifies security requirements. Section 8 provides placeholders for system models and diagrams. Section 9 details use cases. Section 10 documents all configurable properties. Section 11 defines deployment requirements. Section 12 defines the testing strategy and test cases. Section 13 documents project risks. Section 14 documents the future enhancement roadmap. Section 15 contains appendices, including a glossary and standards references.

2 Overall Description

2.1 Product Perspective

The Context-Aware Dynamic Token Mapper is not a standalone product; it is an extension of the Keycloak IAM platform, implemented strictly through Keycloak's supported extension mechanism (the Protocol Mapper SPI). It does not replace, fork, or modify Keycloak's authentication engine, database schema, or Admin Console. The system introduces a new, optional Protocol Mapper type that administrators can attach to a client's token mappers, and a companion external microservice — the AI Risk Assessment Service — that the mapper consults synchronously at token-building time.

As established in the Concept Note, the system preserves a strict separation of concerns across three components: Keycloak Core remains solely responsible for authentication, session management, identity storage, and token signing; the Custom Protocol Mapper is solely responsible for context collection, AI communication, and claim transformation; and the AI Risk Assessment Service is solely responsible for contextual risk evaluation. No component assumes a responsibility belonging to another, and this SRS treats any requirement that would blur this boundary as out of scope.

2.2 Product Functions

- Collect contextual authentication signals from the active Keycloak authentication/user session at token-building time.
- Validate the collected context against a defined schema before transmission.
- Transmit the validated context to the external AI Risk Assessment Service over a synchronous REST call.
- Receive and validate a risk classification response (LOW / MEDIUM / HIGH) from the AI Risk Assessment Service.
- Apply configurable claim transformations to the in-memory token under construction: suppress specified scopes/roles, insert a `risk_level` claim, and insert a restricted claim, according to the returned risk classification.
- Apply a deterministic fallback claim state when the AI Risk Assessment Service is unreachable, times out, or returns an invalid response.
- Expose administrator-configurable properties through the Keycloak Admin Console (endpoint, timeout, retry count, risk threshold, claims to add/remove, cache duration, logging level).
- Log mapper activity, AI Risk Assessment Service interactions, and fallback events for observability and audit purposes.

2.3 User Classes and Characteristics

User Class	Characteristics	Interaction with the System
End User	The individual authenticating against a Keycloak-secured client application. Has no direct knowledge of, or interaction with, the mapper.	Indirectly affected: receives a token whose claims may be adjusted based on assessed risk.
Keycloak Realm Administrator	IAM administrator responsible for configuring realms, clients, and protocol mappers. Technically proficient with Keycloak's Admin Console.	Configures the Custom Protocol Mapper instance on a client, including AI endpoint, thresholds, and claim rules.
System / DevOps Engineer	Responsible for deploying and operating the Keycloak instance and the AI Risk Assessment Service (e.g., via Docker Compose).	Deploys, monitors, and maintains both components; consults logs and health endpoints.
Developer / Maintainer	Software engineer extending or maintaining the mapper or the AI Risk Assessment Service.	Reads this SRS to understand required behavior; implements, tests, and documents changes accordingly.
Security Reviewer / Auditor	Individual responsible for verifying that the extension does not compromise Keycloak's security posture.	Reviews security requirements (Section 7) and audit logs; verifies that persisted identity data is never modified.

2.4 Operating Environment

- Keycloak open-source IAM server (a recent stable major version supporting the Protocol Mapper SPI used by this project) running on the JVM.
- Java Development Kit compatible with the target Keycloak version, used to build and package the custom Protocol Mapper as a provider JAR.
- Apache Maven for building, testing, and packaging the mapper JAR.
- Python 3 runtime hosting the AI Risk Assessment Service, implemented with FastAPI and served via Uvicorn.
- Docker and Docker Compose, used to orchestrate a local/self-hosted development and demonstration environment comprising Keycloak, its backing database, and the AI Risk Assessment Service.
- A network path (HTTP, optionally HTTPS) between the Keycloak host and the AI Risk Assessment Service host, whether co-located on the same Docker network or reachable over a routed network.

2.5 Design Constraints

- The mapper must be implemented strictly by extending Keycloak's AbstractOIDCProtocolMapper and related SPI classes; core Keycloak source code must not be modified.
- The mapper must never write to, or otherwise modify, Keycloak's persistence layer (database) under any circumstance, including risk evaluation, fallback, or error handling paths.
- The AI Risk Assessment Service must never access Keycloak's database, admin API, or any persisted identity data; it may only receive contextual data passed to it over REST.

- The mapper must not perform AI inference internally; all risk computation must occur exclusively within the external AI Risk Assessment Service.
- All communication between the mapper and the AI Risk Assessment Service must occur over REST/HTTP(S) using JSON payloads; no other transport or serialization mechanism is permitted for this integration.
- Every login session must be evaluated independently; no requirement may introduce cross-session state that affects the risk evaluation of another session.
- Version 1 must implement only a rule-based risk engine; machine learning components are explicitly deferred to later versions (see Section 14).

2.6 Assumptions

- The target Keycloak version exposes stable SPI extension points sufficient to intercept the token-building phase for ID Tokens and/or Access Tokens.
- The Keycloak authentication/user session exposes sufficient contextual attributes (IP address, timestamp, user-agent, and, where available, login history) without requiring changes to Keycloak's core session model.
- The AI Risk Assessment Service can be deployed and reached over the network from the Keycloak host within the development environment.
- A rule-based scoring function is an acceptable and sufficient basis for validating the end-to-end architecture in Version 1.
- The development and demonstration environment is a local/self-hosted, non-production Keycloak instance suitable for functional and security testing at a proof-of-concept scale.

2.7 Dependencies

Dependency	Nature	Impact if Unavailable
Keycloak SPI / Protocol Mapper framework	Platform extension point	The mapper cannot be registered or invoked; without it, the entire system is non-functional.
AI Risk Assessment Service availability	External synchronous REST dependency	Token issuance proceeds using the documented fallback claim state (see FR-Fallback requirements); the system degrades gracefully rather than failing authentication.
Docker / Docker Compose runtime	Development and demonstration environment	Local end-to-end demonstration cannot be run; does not affect a manually configured, non-containerized deployment.
Network connectivity between mapper and AI service	Infrastructure	Every request falls back to the default claim state until connectivity is restored; behavior identical to AI service unavailability.
Java/Maven build toolchain	Build-time dependency	The mapper provider JAR cannot be built or packaged for deployment.

3 System Features

This section presents the system's functional areas at a feature level, consistent with the three-component architecture defined in the Concept Note. Each feature is elaborated in Section 4 into individually identified functional requirements. Priority values follow a three-tier scale: Critical (required for the architecture to function at all), High (required for the Version 1 acceptance criteria), and Medium (improves robustness or operability but does not block core functionality).

3.1 Contextual Authentication Data Collection

Attribute	Detail
Description	The Custom Protocol Mapper collects contextual signals from the active Keycloak authentication/user session at the moment a token is being built, without querying any external system or persisted store beyond what the session already exposes.
Priority	Critical
Related Functional Requirements	FR-001 to FR-005
Preconditions	A user has completed standard Keycloak authentication (credential validation and any configured MFA); the token-building phase for the session has begun; the custom mapper is configured and attached to the client's protocol mappers.
Postconditions	A structured context object containing the available contextual attributes for the current session has been assembled in memory.
Acceptance Criteria	For a given test session, the assembled context object contains the IP address, timestamp, and user-agent string recorded for that session, and omits any attribute that is genuinely unavailable rather than substituting a fabricated value.

3.2 Context Validation

Attribute	Detail
Description	Before transmission to the AI Risk Assessment Service, the collected context is validated against a defined schema to ensure it is well-formed and does not contain missing mandatory fields or malformed values.
Priority	Critical
Related Functional Requirements	FR-006 to FR-009
Preconditions	A context object has been assembled by the collection function.
Postconditions	The context object is confirmed valid and ready for transmission, or a validation failure is raised and handled via the fallback path.
Acceptance Criteria	A context object missing a mandatory field, or containing a malformed value (e.g., a non-parsable timestamp), is rejected by the validation step and routed to fallback behavior rather than transmitted.

3.3 AI Risk Assessment Service Communication

Attribute	Detail
Description	The mapper communicates with the external AI Risk Assessment Service exclusively via a synchronous REST/HTTP(S) call carrying a JSON payload, and never performs risk computation itself.
Priority	Critical
Related Functional Requirements	FR-010 to FR-014
Preconditions	A validated context object exists; the AI Risk Assessment Service endpoint is configured.
Postconditions	A risk classification response has been received and validated, or the call has failed/timed out and control has passed to fallback handling.
Acceptance Criteria	A REST call is issued containing the validated context as a JSON body; the mapper never instantiates or executes any local scoring/inference logic under any configuration.

3.4 Risk Evaluation (AI Risk Assessment Service)

Attribute	Detail
Description	The independent AI Risk Assessment Service receives contextual data, validates it, applies a rule-based scoring function, and returns a JSON response containing a risk classification.
Priority	Critical
Related Functional Requirements	FR-015 to FR-018
Preconditions	A well-formed JSON context payload has been received by the AI Risk Assessment Service over its REST endpoint.
Postconditions	A risk classification (LOW, MEDIUM, or HIGH) and, optionally, recommended restrictions, have been returned as a JSON response.
Acceptance Criteria	For an identical, repeated context payload under an unchanged rule set, the AI Risk Assessment Service returns the same risk classification, demonstrating deterministic rule-based behavior.

3.5 Risk-Based Token Claim Transformation

Attribute	Detail
Description	Based on the risk classification returned by the AI Risk Assessment Service, the mapper applies configurable, deterministic transformations to the in-memory token under construction: suppression of specified privileged scopes/roles, insertion of a risk_level claim, and insertion of a restricted claim.
Priority	Critical
Related Functional Requirements	FR-019 to FR-030
Preconditions	A risk classification has been obtained (either from the AI Risk Assessment Service or from fallback behavior).

Attribute	Detail
Postconditions	The in-memory token claim set reflects the configured transformation for the resulting risk classification; no persisted user data has been altered.
Acceptance Criteria	For a HIGH risk classification, the configured privileged scopes/roles are absent from the issued token, and the token contains risk_level=HIGH and restricted=true, while the user's stored roles in the Keycloak database remain unchanged before and after the login.

3.6 Resilience and Fallback Handling

Attribute	Detail
Description	The mapper implements timeout handling, retry logic, and a deterministic fallback claim state to ensure that unavailability or slowness of the AI Risk Assessment Service does not block or destabilize the token-issuance path.
Priority	High
Related Functional Requirements	FR-039 to FR-045
Preconditions	A REST call to the AI Risk Assessment Service has been issued.
Postconditions	Either a valid risk classification has been obtained within the configured timeout and retry budget, or the deterministic fallback claim state has been applied.
Acceptance Criteria	When the AI Risk Assessment Service is stopped, a login still completes successfully within the configured timeout and retry budget, and the issued token contains the documented fallback claim state.

3.7 Configuration Management

Attribute	Detail
Description	All operationally significant properties of the mapper (AI endpoint, timeout, retry count, risk threshold, claims to add/remove, cache duration, logging level, connection settings) are exposed as configuration properties in the Keycloak Admin Console, without requiring code changes or redeployment.
Priority	High
Related Functional Requirements	FR-035 to FR-038, FR-050 to FR-051
Preconditions	The mapper provider JAR has been deployed and registered with Keycloak.
Postconditions	An administrator can view and modify the mapper's configuration through the standard Keycloak Admin Console client mapper configuration screen.
Acceptance Criteria	Changing the configured risk threshold or the AI endpoint through the Admin Console changes mapper behavior on the next login without requiring a Keycloak restart or JAR redeployment.

3.8 Logging and Observability

Attribute	Detail
Description	The mapper and the AI Risk Assessment Service produce structured logs covering context collection, REST communication outcomes, applied claim transformations, and fallback events, at a configurable logging level.
Priority	Medium
Related Functional Requirements	FR-031 to FR-034, FR-046 to FR-047
Preconditions	The mapper is invoked during a token-building phase, or the AI Risk Assessment Service receives a request.
Postconditions	A log entry describing the relevant event has been written at the configured verbosity, without including any information restricted under Section 7 (Security Requirements).
Acceptance Criteria	With logging level set to DEBUG, a test login produces log entries for context collection, the outbound REST call, the received (or fallback) risk classification, and the applied claim transformation, none of which contain raw credentials.

4 Functional Requirements

Each functional requirement below is uniquely identified (FR-XXX), grouped by functional category as specified in the project brief, and elaborated with a requirement statement, purpose, rationale, priority, dependencies, and acceptance criteria. Requirement statements use "shall" to denote mandatory behavior, consistent with IEEE 29148 conventions.

4.1 Authentication Context Collection (FR-001 – FR-005)

FR-001 — Collect source IP address

Requirement Statement	The mapper shall extract the source IP address associated with the current authentication session from Keycloak's session data at token-building time.
Purpose	Provides the AI Risk Assessment Service with a network-origin signal for risk evaluation.
Rationale	Network origin is one of the most common contextual risk indicators (e.g., unfamiliar or high-risk IP ranges).
Priority	Critical
Dependencies	Requires the Keycloak authentication session to expose the client IP address; depends on FR-006 (context validation).
Acceptance Criteria	For a test login from a known IP, the assembled context object contains that exact IP address string.

FR-002 — Collect authentication timestamp

Requirement Statement	The mapper shall record the timestamp of the current authentication event, expressed in a consistent, timezone-aware format, as part of the collected context.
Purpose	Enables the AI Risk Assessment Service to evaluate time-of-day or time-based access patterns.
Rationale	Unusual access times relative to a user's typical pattern are a recognized contextual risk indicator.
Priority	Critical
Dependencies	None beyond system clock availability.
Acceptance Criteria	The timestamp recorded in the context object is within one second of the actual authentication event time, in ISO 8601 format.

FR-003 — Collect user-agent / device characteristics

Requirement Statement	The mapper shall extract the user-agent string (and any other readily available device characteristics) from the authentication request associated with the current session.
Purpose	Provides a device/browser signal that can indicate an unfamiliar or atypical client accessing the account.
Rationale	Device/browser fingerprints are a standard, low-cost contextual signal used broadly in adaptive authentication.
Priority	High
Dependencies	Requires the user-agent header to be accessible from the authentication session context.
Acceptance Criteria	The context object's user-agent field matches the User-Agent header sent by the test client exactly.

FR-004 — Collect accessible login history

Requirement Statement	Where accessible via Keycloak session notes or events, the mapper shall include a summary of recent login history (e.g., last successful login timestamp) in the collected context.
Purpose	Allows the AI Risk Assessment Service to consider recency/frequency patterns without requiring a new persisted data store.
Rationale	Login recency is a lightweight behavioral signal obtainable without extending Keycloak's data model.
Priority	Medium
Dependencies	Requires that Keycloak session notes/events expose prior login information for the user; degrades gracefully if unavailable.
Acceptance Criteria	When prior login information is available in the session, it is included in the context object; when unavailable, the field is omitted rather than populated with placeholder data.

FR-005 — Assemble structured context object

Requirement Statement	The mapper shall assemble all collected contextual attributes into a single, structured, well-defined context object prior to validation and transmission.
Purpose	Provides a single, coherent unit of data for downstream validation and REST transmission.
Rationale	A single structured object simplifies validation, logging, and testing compared to ad hoc attribute passing.
Priority	Critical
Dependencies	Depends on FR-001 through FR-004.
Acceptance Criteria	The assembled context object conforms to the schema defined in Section 6.4 for every test login scenario.

4.2 Context Validation (FR-006 – FR-009)

FR-006 — Validate mandatory fields present

Requirement Statement	The mapper shall verify that all mandatory fields of the context schema (at minimum, IP address and timestamp) are present before the context is transmitted to the AI Risk Assessment Service.
Purpose	Prevents transmission of incomplete data that could cause the AI Risk Assessment Service to fail or return an unreliable classification.
Rationale	Well-formed input is a precondition for a trustworthy risk evaluation.
Priority	Critical
Dependencies	Depends on FR-005.
Acceptance Criteria	A context object missing the timestamp field is rejected by validation and routed to the fallback path (FR-043) rather than transmitted.

FR-007 — Validate field formats

Requirement Statement	The mapper shall verify that each field in the context object conforms to its expected format (e.g., valid IP address syntax, ISO 8601 timestamp) before transmission.
Purpose	Prevents malformed data from being sent to, or misinterpreted by, the AI Risk Assessment Service.
Rationale	Format validation at the source is more reliable and easier to test than relying solely on the AI Risk Assessment Service to reject bad input.
Priority	High
Dependencies	Depends on FR-005.
Acceptance Criteria	A context object containing a non-parsable timestamp value is rejected locally and never transmitted over REST.

FR-008 — Reject and log invalid context

Requirement Statement	When context validation fails, the mapper shall reject the transmission attempt and record a log entry describing the validation failure, then proceed to fallback behavior.
Purpose	Ensures validation failures are visible for troubleshooting rather than silently ignored.
Rationale	Observability of failure modes is required for maintainability (Section 5) and for the logging requirements (Section 4.10).
Priority	High
Dependencies	Depends on FR-006, FR-007, and FR-031.
Acceptance Criteria	A validation failure produces exactly one log entry identifying the failed field, and the mapper proceeds to apply the fallback claim state.

FR-009 — Schema versioning of context payload

Requirement Statement	The context object shall include a schema version identifier so that the AI Risk Assessment Service can interpret the payload correctly across independent deployments of the two components.
Purpose	Allows the mapper and the AI Risk Assessment Service to evolve independently without silently breaking compatibility.
Rationale	Supports the modular, loosely coupled architecture mandated by the Concept Note.
Priority	Medium
Dependencies	Depends on FR-005.
Acceptance Criteria	The context object transmitted in every test case includes a <code>schema_version</code> field matching a version the AI Risk Assessment Service declares support for.

4.3 REST Communication (FR-010 – FR-014)**FR-010 — Transmit context via synchronous REST call**

Requirement Statement	The mapper shall transmit the validated context object to the configured AI Risk Assessment Service endpoint via a synchronous HTTP(S) POST request with a JSON body.
Purpose	Implements the mandated REST-only integration between the mapper and the AI Risk Assessment Service.
Rationale	The Concept Note requires that the mapper communicate with the AI Risk Assessment Service only through REST, and never perform inference itself.
Priority	Critical
Dependencies	Depends on FR-006 through FR-009 (validated context) and configuration item AI Endpoint (Section 10).
Acceptance Criteria	A network capture of a test login shows exactly one outbound HTTPS/HTTP POST request to the configured endpoint per mapper invocation, carrying the context object as a JSON body.

FR-011 — Await and receive synchronous response

Requirement Statement	The mapper shall await the AI Risk Assessment Service's response synchronously, within the configured timeout window, before proceeding to apply claim transformations.
Purpose	Guarantees that claim transformation decisions are based on a completed risk evaluation, or on a well-defined fallback, never on a partial response.
Rationale	Token issuance must remain deterministic and auditable; asynchronous or fire-and-forget calls would violate this.
Priority	Critical
Dependencies	Depends on FR-010 and configuration item Timeout (Section 10).
Acceptance Criteria	For a response returned within the timeout window, the mapper's claim transformation logic executes only after that response has been fully received and parsed.

FR-012 — Parse and validate the AI response payload

Requirement Statement	The mapper shall parse the JSON response from the AI Risk Assessment Service and validate that it contains a recognized risk classification value before use.
Purpose	Prevents an unexpected or malformed response from producing an unpredictable claim transformation.
Rationale	Defensive parsing is required given that the AI Risk Assessment Service is an independently deployed, externally maintained component.
Priority	Critical
Dependencies	Depends on FR-011.
Acceptance Criteria	A response containing an unrecognized risk classification value (e.g., an unexpected string) is treated as an invalid response and routed to fallback behavior (FR-043).

FR-013 — Secure transport configuration

Requirement Statement	The mapper shall support HTTPS as the transport scheme for communication with the AI Risk Assessment Service, configurable per deployment.
Purpose	Allows the integration to be secured in accordance with the security requirements in Section 7.
Rationale	Contextual data, while not user credentials, may still be sensitive (e.g., IP address) and warrants transport protection where the deployment environment requires it.
Priority	High
Dependencies	Depends on configuration item Connection Settings (Section 10).
Acceptance Criteria	When the AI endpoint is configured with an https:// scheme, all outbound requests use TLS; when configured with http:// in a development environment, the configuration is clearly documented as non-production.

FR-014 — Correlate requests and responses

Requirement Statement	The mapper shall attach a unique correlation identifier to each outbound request to the AI Risk Assessment Service and log it alongside the corresponding response.
Purpose	Enables end-to-end tracing of a single authentication event across both components' logs.
Rationale	Supports observability (Section 5.9) and troubleshooting of intermittent AI Risk Assessment Service issues.
Priority	Medium
Dependencies	Depends on FR-010 and FR-031.
Acceptance Criteria	For any test login, the correlation identifier recorded in the mapper's log matches the one recorded in the AI Risk Assessment Service's log for the same request.

4.4 Risk Evaluation (FR-015 – FR-018)

FR-015 — Receive and validate incoming context (AI Service)

Requirement Statement	The AI Risk Assessment Service shall receive the context payload over its REST endpoint and validate it against the agreed schema before scoring.
Purpose	Ensures the AI Risk Assessment Service does not attempt to score malformed or incomplete input.
Rationale	Defensive input handling is required of any independently deployed service receiving external input.
Priority	Critical
Dependencies	Depends on FR-010 (mapper transmission) and the schema defined in Section 6.4.
Acceptance Criteria	A payload missing a mandatory field is rejected by the AI Risk Assessment Service with a documented error response rather than silently scored.

FR-016 — Apply rule-based scoring function

Requirement Statement	The AI Risk Assessment Service shall compute a risk classification for the received context using a deterministic, rule-based scoring function, as defined for Version 1 in the Concept Note's AI Roadmap.
Purpose	Implements the Version 1 scope: a rule-based engine sufficient to validate the architecture, deferring machine learning to later versions.
Rationale	The Concept Note explicitly scopes Version 1 to a rule-based engine to keep focus on architecture and integration rather than model sophistication.
Priority	Critical
Dependencies	Depends on FR-015.
Acceptance Criteria	For a fixed context payload and an unchanged rule configuration, the AI Risk Assessment Service returns an identical risk classification across repeated calls.

FR-017 — Return structured JSON risk response

Requirement Statement	The AI Risk Assessment Service shall return a structured JSON response containing, at minimum, the assessed risk classification, and optionally, recommended restrictions.
Purpose	Provides the mapper with a machine-readable, schema-conformant response it can parse deterministically.
Rationale	A structured, versioned response contract is required to keep the two components loosely coupled.
Priority	Critical
Dependencies	Depends on FR-016 and the response schema defined in Section 6.4.
Acceptance Criteria	The response body for every scoring request conforms to the JSON schema documented in Section 6.4, including a <code>risk_classification</code> field with one of the three defined values.

FR-018 — Expose scoring rule configuration

Requirement Statement	The AI Risk Assessment Service shall allow its rule-based scoring thresholds to be configured (e.g., via configuration file or environment variables) without requiring code changes.
Purpose	Allows the scoring behavior to be tuned or adjusted for testing and demonstration without redeployment of new code.
Rationale	Supports maintainability and testability requirements (Section 5.4, Section 5.10).
Priority	Medium
Dependencies	None beyond the service's own configuration loading mechanism.
Acceptance Criteria	Changing a documented threshold in the AI Risk Assessment Service's configuration changes the classification boundary observed in test cases without a code change.

4.5 Token Claim Modification (FR-019 – FR-021)**FR-019 — Modify only the in-memory token under construction**

Requirement Statement	The mapper shall apply all claim transformations exclusively to the in-memory token representation currently being built for the active session, and shall never modify Keycloak's persisted user, role, or group data.
Purpose	Preserves the non-destructive, session-scoped nature of the extension mandated by the Concept Note.
Rationale	This is a foundational architectural principle: the user's stored identity record must remain unaffected by risk-based decisions.
Priority	Critical
Dependencies	Depends on the token-building extension point exposed by the Keycloak SPI.
Acceptance Criteria	A database comparison of the user's stored roles, groups, and attributes taken immediately before and after a HIGH-risk login shows zero differences.

FR-020 — Apply transformation deterministically per risk classification

Requirement Statement	The mapper shall apply a deterministic, configured claim transformation corresponding to the received (or fallback) risk classification, with no randomization or session-external influence.
Purpose	Ensures predictable, testable, and auditable behavior for a given risk classification.
Rationale	Deterministic behavior is required for the acceptance criteria across Sections 4 and 12 to be verifiable.
Priority	Critical
Dependencies	Depends on FR-012 (or FR-043 for fallback) and configuration items Claims To Remove / Claims To Add (Section 10).
Acceptance Criteria	Given the same risk classification and the same mapper configuration, the resulting claim set is identical across repeated test logins.

FR-021 — Apply transformation before token signing

Requirement Statement	The mapper shall complete all claim transformations before Keycloak proceeds to sign the token.
Purpose	Ensures the signed token accurately reflects the risk-based decision; a transformation applied after signing would be architecturally invalid and cryptographically inconsistent.
Rationale	Aligns with Keycloak's documented token-building lifecycle, in which Protocol Mappers execute prior to signing.
Priority	Critical
Dependencies	Depends on the Keycloak Protocol Mapper invocation order during token building.
Acceptance Criteria	Decoding the signed token for every test case shows the expected claim set, with no discrepancy between pre-signing and post-signing claims.

4.6 Scope Suppression (FR-022 – FR-024)**FR-022 — Suppress configured privileged scopes**

Requirement Statement	For sessions classified at or above the configured risk threshold, the mapper shall remove the administrator-configured set of privileged scopes from the token's scope claim.
Purpose	Reduces the effective privilege of a session assessed as risky, without altering the user's stored scope entitlements.
Rationale	Directly implements the Concept Note's requirement for 'removal or suppression of specified privileged scopes or roles' for high-risk sessions.
Priority	Critical
Dependencies	Depends on FR-020 and configuration item Claims To Remove (Section 10).
Acceptance Criteria	For a HIGH-risk test login with 'admin:write' configured as a suppressed scope, the issued token's scope claim does not contain 'admin:write', while a LOW-risk login for the same user retains it.

FR-023 — Preserve non-configured scopes

Requirement Statement	The mapper shall leave all scopes not explicitly configured for suppression unchanged in the issued token.
Purpose	Prevents the mapper from over-restricting sessions beyond its configured, intended behavior.
Rationale	Minimizes unintended side effects and keeps the transformation narrowly scoped and predictable.
Priority	High
Dependencies	Depends on FR-022.
Acceptance Criteria	A scope not listed in the Claims To Remove configuration is present in the issued token regardless of the session's risk classification.

FR-024 — Support per-client scope suppression configuration

Requirement Statement	The mapper shall allow the set of suppressible scopes to be configured independently for each client on which it is attached.
Purpose	Allows different applications to define different privileged-scope policies appropriate to their own risk tolerance.
Rationale	Supports the configurability and extensibility principles mandated by the Concept Note.
Priority	Medium
Dependencies	Depends on the Keycloak client-level Protocol Mapper configuration mechanism.
Acceptance Criteria	Two clients with different Claims To Remove configurations produce different suppressed-scope sets for an identical HIGH-risk user session.

4.7 Role Suppression (FR-025 – FR-026)**FR-025 — Suppress configured privileged roles from token claims**

Requirement Statement	For sessions classified at or above the configured risk threshold, the mapper shall remove the administrator-configured set of privileged roles from the token's role claim(s), without altering the user's stored role assignments.
Purpose	Reduces effective, token-level privilege for risky sessions while leaving the identity record intact.
Rationale	Directly implements the Concept Note's requirement to suppress roles from the issued token without permanent alteration.
Priority	Critical
Dependencies	Depends on FR-020 and configuration item Claims To Remove (Section 10).
Acceptance Criteria	For a HIGH-risk login of a user holding the 'realm-admin' role, the issued token's role claim omits 'realm-admin', while the Keycloak Admin Console still shows the role assigned to the user.

FR-026 — Verify persisted role assignments remain unchanged

Requirement Statement	The system shall be independently verifiable, through the Keycloak Admin Console or Admin REST API, to show unchanged role assignments for a user regardless of the risk classification applied during any login.
Purpose	Provides an auditable guarantee that role suppression is strictly session-scoped.
Rationale	This verification directly satisfies Specific Objective 7 of the Concept Note.
Priority	Critical
Dependencies	Depends on FR-019 and FR-025.
Acceptance Criteria	A before/after comparison of a user's assigned roles via the Admin Console, taken across a LOW-risk and a subsequent HIGH-risk login, shows no difference.

4.8 Risk Claim Insertion (FR-027 – FR-028)

FR-027 — Insert risk_level claim

Requirement Statement	The mapper shall insert a risk_level claim into the issued token, with a value corresponding to the risk classification applied to the session (LOW, MEDIUM, or HIGH).
Purpose	Communicates the assessed risk classification to downstream applications so they may apply their own risk-aware logic if desired.
Rationale	Directly implements the Concept Note's requirement for a risk_level claim reflecting the assessed classification.
Priority	Critical
Dependencies	Depends on FR-020.
Acceptance Criteria	Decoding the token for a MEDIUM-risk test login shows a risk_level claim with the exact value 'MEDIUM'.

FR-028 — Make risk_level claim name configurable

Requirement Statement	The mapper shall allow the name of the inserted risk claim to be configured, defaulting to risk_level if not otherwise specified.
Purpose	Avoids claim name collisions with existing claims used by client applications.
Rationale	Supports configurability and safe integration into environments with pre-existing claim naming conventions.
Priority	Medium
Dependencies	Depends on FR-027.
Acceptance Criteria	When the claim name is configured as 'auth_risk', the issued token contains that claim name instead of 'risk_level', with the correct value.

4.9 Restricted Claim (FR-029 – FR-030)

FR-029 — Insert restricted claim

Requirement Statement	The mapper shall insert a restricted claim into the issued token, set to true when the session's risk classification meets or exceeds the configured threshold, and false otherwise.
Purpose	Provides a simple, boolean signal that downstream applications can use to gate sensitive functionality without needing to interpret the full risk_level value.
Rationale	Directly implements the Concept Note's requirement for a restricted boolean/status claim.
Priority	Critical
Dependencies	Depends on FR-020 and configuration item Risk Threshold (Section 10).
Acceptance Criteria	For a HIGH-risk test login exceeding the configured threshold, the issued token contains restricted=true; for a LOW-risk login, it contains restricted=false.

FR-030 — Configurable restricted claim name and type

Requirement Statement	The mapper shall allow the restricted claim's name to be configured, and shall support boolean representation as the default value type.
Purpose	Avoids naming collisions and allows integration flexibility across different client applications.
Rationale	Supports configurability principles mandated by the Concept Note.
Priority	Medium
Dependencies	Depends on FR-029.
Acceptance Criteria	When configured with claim name 'reduced_privilege', the issued token contains that claim name with a boolean value reflecting the session's risk status.

4.10 Logging (FR-031 – FR-034)**FR-031 — Log mapper lifecycle events**

Requirement Statement	The mapper shall log, at a configurable verbosity, the start and completion of context collection, the outbound AI Risk Assessment Service call, and the applied claim transformation for each invocation.
Purpose	Provides operational visibility into the mapper's behavior for troubleshooting and audit.
Rationale	Directly implements the Concept Note's requirement for logging as a mapper responsibility.
Priority	High
Dependencies	Depends on configuration item Logging Level (Section 10).
Acceptance Criteria	With logging level set to INFO, a single test login produces log entries for context collection start, AI call outcome, and applied transformation, in that order.

FR-032 — Exclude sensitive data from logs

Requirement Statement	The mapper and the AI Risk Assessment Service shall exclude raw user credentials and full token signatures from all log output, regardless of configured logging level.
Purpose	Prevents sensitive authentication material from being exposed through log files.
Rationale	Directly required by the security requirements in Section 7 (Logging Security).
Priority	Critical
Dependencies	Depends on FR-031.
Acceptance Criteria	A review of DEBUG-level log output across all test cases confirms the absence of raw passwords, full signed JWTs, or private key material.

FR-033 — Log fallback and error events distinctly

Requirement Statement	The mapper shall log fallback activations, timeouts, retries, and validation failures as distinctly identifiable log entries, separate from normal-path log entries.
Purpose	Allows operators to quickly identify degraded-mode operation without parsing verbose normal-path logs.
Rationale	Supports observability requirements (Section 5.9) and operational maintainability.
Priority	High
Dependencies	Depends on FR-031, FR-039 through FR-045.
Acceptance Criteria	A log search for the fallback event tag returns every test case in which the AI Risk Assessment Service was intentionally made unavailable, and no others.

FR-034 — Log AI Risk Assessment Service scoring decisions

Requirement Statement	The AI Risk Assessment Service shall log each received context (excluding any field designated sensitive) and the corresponding risk classification returned.
Purpose	Enables independent verification that the scoring function behaved as expected for a given input.
Rationale	Supports testability (Section 5.10) and auditability of the risk-scoring component.
Priority	Medium
Dependencies	Depends on FR-015 through FR-017.
Acceptance Criteria	For each of the 30+ test cases in Section 12, the AI Risk Assessment Service's log contains a corresponding entry with the received context summary and returned classification.

4.11 Configuration (FR-035 – FR-038)**FR-035 — Expose configuration via Keycloak Admin Console**

Requirement Statement	The mapper shall expose all properties listed in Section 10 (Configuration Requirements) as configurable fields on its Protocol Mapper configuration screen within the Keycloak Admin Console.
Purpose	Allows administrators to configure the mapper using standard Keycloak tooling, without editing code or configuration files directly.
Rationale	Consistent with Keycloak's own configuration conventions for Protocol Mappers, minimizing the learning curve for administrators.
Priority	Critical
Dependencies	Depends on the Keycloak Admin Console's Protocol Mapper configuration extension mechanism.
Acceptance Criteria	Every configuration property listed in Section 10 is visible and editable on the mapper's configuration screen in the Admin Console.

FR-036 — Validate configuration values on save

Requirement Statement	The Admin Console configuration screen shall validate mapper configuration values (e.g., a numeric timeout, a well-formed endpoint URL) at the time an administrator saves the configuration.
Purpose	Prevents invalid configuration from being persisted and causing runtime failures.
Rationale	Improves usability and reduces misconfiguration risk for administrators unfamiliar with the mapper's internals.
Priority	Medium
Dependencies	Depends on FR-035.
Acceptance Criteria	Attempting to save a non-numeric value into the Timeout field is rejected by the Admin Console with a descriptive validation message.

FR-037 — Apply configuration changes without restart

Requirement Statement	Configuration changes made through the Admin Console shall take effect on the next authentication request without requiring a restart of the Keycloak server or redeployment of the mapper JAR.
Purpose	Allows administrators to tune mapper behavior operationally, without service interruption.
Rationale	Supports maintainability and configurability requirements (Section 5.4, Section 5.12).
Priority	High
Dependencies	Depends on FR-035 and Keycloak's standard configuration reload mechanism for Protocol Mappers.
Acceptance Criteria	Changing the Risk Threshold value and immediately performing a new login reflects the updated threshold in that login's outcome.

FR-038 — Provide sensible default configuration values

Requirement Statement	The mapper shall define sensible default values for every configuration property, such that it can be deployed and produce safe, documented behavior even before an administrator customizes it.
Purpose	Reduces the risk of an insecure or non-functional default deployment.
Rationale	Supports usability (Section 5.13) and safe-by-default operation.
Priority	High
Dependencies	Depends on FR-035.
Acceptance Criteria	A fresh deployment of the mapper with no administrator customization applies the documented default fallback claim state and default timeout without error.

4.12 Timeout Handling (FR-039 – FR-040)

FR-039 — Enforce configurable request timeout

Requirement Statement	The mapper shall enforce a configurable maximum wait time for the AI Risk Assessment Service's response, after which the request is treated as failed.
Purpose	Prevents an unresponsive AI Risk Assessment Service from indefinitely blocking token issuance.
Rationale	Directly implements the Concept Note's requirement for explicit, documented timeout handling.
Priority	Critical
Dependencies	Depends on configuration item Timeout (Section 10).
Acceptance Criteria	With Timeout configured to 500 milliseconds and the AI Risk Assessment Service artificially delayed by 2 seconds, the mapper treats the call as timed out at approximately 500 milliseconds and proceeds to fallback or retry.

FR-040 — Distinguish timeout from other failure types

Requirement Statement	The mapper shall distinguish a timeout from a connection failure or an invalid response, and shall log each failure type distinctly.
Purpose	Enables operators to diagnose whether an issue is network latency, service unavailability, or a response-contract mismatch.
Rationale	Supports observability (Section 5.9) and precise fallback/retry decision-making.
Priority	Medium
Dependencies	Depends on FR-039, FR-033.
Acceptance Criteria	Test cases simulating a timeout, a connection refusal, and a malformed response each produce distinctly labeled log entries.

4.13 Retry Logic (FR-041 – FR-042)

FR-041 — Retry failed requests up to a configured count

Requirement Statement	Upon a timeout or transient connection failure, the mapper shall retry the request to the AI Risk Assessment Service up to a configurable maximum retry count before invoking fallback behavior.
Purpose	Improves resilience against transient network or service issues without resorting immediately to the fallback claim state.
Rationale	Directly implements the Concept Note's requirement for resilient integration handling.
Priority	High
Dependencies	Depends on configuration item Retry Count (Section 10) and FR-039.
Acceptance Criteria	With Retry Count configured to 2 and the AI Risk Assessment Service failing the first two attempts then succeeding on the third, the mapper applies the risk classification from the successful third attempt.

FR-042 — Bound total retry duration within the token-building path

Requirement Statement	The cumulative time spent across all retries and timeouts for a single mapper invocation shall not exceed a documented maximum bound, to protect overall login responsiveness.
Purpose	Prevents the retry mechanism itself from becoming a source of unacceptable authentication latency.
Rationale	Supports the performance requirements defined in Section 5.1.
Priority	High
Dependencies	Depends on FR-039, FR-041, and NFR-001.
Acceptance Criteria	With Timeout=500ms and Retry Count=2, the total time added to the login path by AI Risk Assessment Service communication does not exceed approximately 1.5 seconds in the worst case.

4.14 Fallback Behaviour (FR-043 – FR-045)**FR-043 — Apply deterministic fallback claim state**

Requirement Statement	When the AI Risk Assessment Service is unreachable, times out after all retries, or returns an invalid response, the mapper shall apply a predefined, documented, deterministic fallback claim state to the token.
Purpose	Ensures token issuance completes predictably and safely even when the external dependency is unavailable.
Rationale	Directly implements the Concept Note's explicit requirement for documented fallback behavior given the AI service's synchronous, external nature.
Priority	Critical
Dependencies	Depends on FR-039 through FR-042.
Acceptance Criteria	With the AI Risk Assessment Service stopped, a test login still completes successfully and the issued token contains the documented fallback risk_level and restricted claim values.

FR-044 — Make fallback claim state configurable

Requirement Statement	The specific fallback claim state (e.g., a conservative HIGH/restricted state, or an unmodified/default claim set) shall be configurable by the administrator.
Purpose	Allows organizations to choose between a conservative (fail-closed) or permissive (fail-open) posture according to their own risk appetite.
Rationale	Supports configurability and reflects that different deployments may reasonably choose different fallback postures.
Priority	High
Dependencies	Depends on FR-043.
Acceptance Criteria	Configuring the fallback posture as fail-closed produces restricted=true and risk_level=HIGH when the AI Risk Assessment Service is unavailable; configuring it as fail-open produces an unmodified claim set under the same condition.

FR-045 — Never fail authentication due to AI Risk Assessment Service unavailability

Requirement Statement	Unavailability, timeout, or an invalid response from the AI Risk Assessment Service shall never, by itself, cause authentication or token issuance to fail.
Purpose	Guarantees that the extension does not introduce a new single point of failure into the core authentication path.
Rationale	Directly supports the availability requirements in Section 5.3 and preserves Keycloak's own authentication reliability.
Priority	Critical
Dependencies	Depends on FR-043.
Acceptance Criteria	Across all fallback-triggering test cases in Section 12, the login flow completes with a valid, signed token in 100% of attempts.

4.15 Health Checking (FR-046 – FR-047)**FR-046 — AI Risk Assessment Service exposes a health endpoint**

Requirement Statement	The AI Risk Assessment Service shall expose a lightweight REST health-check endpoint that reports its operational status independently of the scoring endpoint.
Purpose	Allows operators and monitoring tooling to verify service availability without invoking a full scoring request.
Rationale	Standard practice for independently deployable microservices, supporting observability and availability requirements.
Priority	Medium
Dependencies	None.
Acceptance Criteria	A GET request to the health endpoint returns a success status and a documented body when the service is running normally.

FR-047 — Mapper-side connectivity check on startup (optional)

Requirement Statement	The mapper may optionally perform a non-blocking connectivity check to the configured AI Risk Assessment Service endpoint when Keycloak starts, logging the result without blocking Keycloak startup.
Purpose	Provides early visibility into misconfiguration (e.g., an unreachable endpoint) without affecting Keycloak's own startup sequence.
Rationale	Improves operability during deployment and configuration changes.
Priority	Medium
Dependencies	Depends on FR-046 and FR-035.
Acceptance Criteria	When the configured endpoint is unreachable at Keycloak startup, a warning log entry is produced during startup, and Keycloak startup completes without delay or failure.

4.16 Deployment (FR-048 – FR-049)

FR-048 — Package mapper as a deployable provider JAR

Requirement Statement	The mapper shall be packaged as a standard Keycloak SPI provider JAR, deployable by placing it in Keycloak's providers directory and running the standard provider registration process.
Purpose	Ensures the mapper can be deployed using Keycloak's own supported extension mechanism, without custom deployment tooling.
Rationale	Directly required by the Concept Note's specification that the mapper be 'deployed as a Keycloak provider JAR.'
Priority	Critical
Dependencies	Depends on the Maven build configuration and Keycloak's SPI provider discovery mechanism.
Acceptance Criteria	Placing the built JAR into the providers directory and running Keycloak's provider build step makes the mapper selectable in the Admin Console's Protocol Mapper type list.

FR-049 — Provide a local Docker Compose demonstration environment

Requirement Statement	The project shall provide a Docker Compose configuration that stands up Keycloak (with the mapper deployed), its backing database, and the AI Risk Assessment Service, sufficient for an end-to-end demonstration.
Purpose	Enables reproducible local testing and demonstration without manual multi-step environment setup.
Rationale	Directly required by the Concept Note's expected deliverable of a local/demo deployment setup.
Priority	High
Dependencies	Depends on FR-048 and the AI Risk Assessment Service's own containerization.
Acceptance Criteria	Running the documented docker compose up command results in a fully functional environment in which an end-to-end login demonstrates claim transformation, without additional manual configuration steps beyond those documented.

4.17 Administration (FR-050 – FR-051)

FR-050 — Provide administrator-facing configuration documentation

Requirement Statement	The project shall provide documentation describing every configuration property, its purpose, valid values, and default value, sufficient for an administrator to configure the mapper without reading its source code.
Purpose	Supports usability and maintainability for administrators who are not the original developers.
Rationale	Directly required by the Concept Note's expected deliverable of configuration and operational documentation.
Priority	High
Dependencies	Depends on Section 10 (Configuration Requirements).

Acceptance Criteria	An administrator unfamiliar with the codebase can, using only the provided documentation, correctly configure the AI endpoint, timeout, and risk threshold on a test client.
----------------------------	--

FR-051 — Support enabling and disabling the mapper without side effects

Requirement Statement	An administrator shall be able to remove or disable the Custom Protocol Mapper from a client's configuration at any time, with the immediate effect that tokens for that client are issued without any risk-based claim transformation, and with no residual effect on previously issued tokens or stored user data.
Purpose	Ensures the mechanism can be safely piloted, rolled back, or disabled, consistent with the Concept Note's emphasis on safe, reversible, non-destructive design.
Rationale	Directly required by the Concept Note's statement that the mechanism can be 'safely disabled or reconfigured without any data migration or cleanup.'
Priority	Critical
Dependencies	Depends on FR-019 and standard Keycloak client mapper management.
Acceptance Criteria	Removing the mapper from a test client and performing a new login produces a token with the standard, unmodified claim set, with no error and no residual risk_level or restricted claims.

5 Non-Functional Requirements

Non-functional requirements (NFR) constrain how the system performs its functions rather than what functions it performs. Each is uniquely identified (NFR-XXX) and grouped by quality attribute.

5.1 Performance

ID	Requirement
NFR-001	The mapper shall add no more than 1.5 seconds of latency to the token-issuance path under normal AI Risk Assessment Service response conditions (excluding fallback/retry scenarios), measured at the 95th percentile.
NFR-002	The AI Risk Assessment Service shall return a scoring response within 300 milliseconds under normal load for a rule-based evaluation, measured at the 95th percentile.

5.2 Reliability

ID	Requirement
NFR-003	The mapper shall complete token issuance successfully in 100% of test cases regardless of AI Risk Assessment Service availability, by invoking documented fallback behavior when necessary.
NFR-004	The AI Risk Assessment Service shall not crash or become unresponsive when receiving malformed input; it shall return a documented error response instead.

5.3 Availability

ID	Requirement
NFR-005	Unavailability of the AI Risk Assessment Service shall not reduce the availability of Keycloak's core authentication service, consistent with FR-045.
NFR-006	The AI Risk Assessment Service, when deployed via the provided Docker Compose configuration, shall automatically restart on failure within the development environment.

5.4 Maintainability

ID	Requirement
NFR-007	The mapper's context-collection, REST-communication, and claim-transformation responsibilities shall be implemented as separately testable units (classes/modules) to support independent maintenance.
NFR-008	Configuration properties documented in Section 10 shall be centrally defined in a single configuration provider class, avoiding scattered hard-coded values.

5.5 Scalability

ID	Requirement
NFR-009	The AI Risk Assessment Service shall be independently horizontally scalable (i.e., runnable as multiple stateless instances behind a load balancer) without requiring changes to the mapper.
NFR-010	The mapper's per-invocation resource usage shall not grow with the number of concurrent Keycloak sessions beyond what Keycloak's own session handling already requires.

5.6 Security

ID	Requirement
NFR-011	All communication between the mapper and the AI Risk Assessment Service shall support TLS encryption when deployed outside a fully trusted, isolated network.
NFR-012	The mapper shall never transmit user credentials, full token contents, or private key material to the AI Risk Assessment Service.
NFR-013	The AI Risk Assessment Service shall validate and sanitize all incoming request fields to prevent injection or malformed-payload-based failures.

5.7 Extensibility

ID	Requirement
NFR-014	The claim-transformation logic shall be structured so that additional claim types can be added in future versions without modifying the context-collection or REST-communication components.
NFR-015	The AI Risk Assessment Service's scoring function shall be replaceable (e.g., rule-based to machine-learning-based) without requiring any change to the mapper's REST contract, consistent with the AI Roadmap in Section 14.

5.8 Configurability

ID	Requirement
NFR-016	Every property listed in Section 10 shall be configurable without modifying or recompiling source code.
NFR-017	Default configuration values shall be safe (i.e., fail-closed or clearly documented as fail-open) for a first-time deployment.

5.9 Observability

ID	Requirement
NFR-018	The mapper and the AI Risk Assessment Service shall emit structured, timestamped log entries sufficient to reconstruct the sequence of events for any given authentication attempt.
NFR-019	Fallback activations, retries, and timeouts shall be individually countable from log output to support operational monitoring.

5.10 Testability

ID	Requirement
NFR-020	The mapper's claim-transformation logic shall be unit-testable independently of a running Keycloak server, using mocked session and AI-response inputs.
NFR-021	The AI Risk Assessment Service's scoring function shall be unit-testable independently of the REST layer.

5.11 Portability

ID	Requirement
NFR-022	The mapper shall be deployable on any Keycloak instance meeting the documented version and JVM requirements, without dependency on a specific host operating system.
NFR-023	The AI Risk Assessment Service shall run in any environment providing a supported Python runtime and shall not depend on any Keycloak-specific library.

5.12 Interoperability

ID	Requirement
NFR-024	The mapper shall produce standards-compliant OIDC tokens (valid JWT structure and required standard claims) regardless of the applied risk-based transformation.
NFR-025	The REST contract between the mapper and the AI Risk Assessment Service shall use only JSON and standard HTTP status codes, avoiding proprietary or non-standard transport mechanisms.

5.13 Usability

ID	Requirement
NFR-026	Configuration fields exposed in the Keycloak Admin Console shall include descriptive labels and help text sufficient for an administrator to configure the mapper without external documentation for common cases.
NFR-027	Error and fallback log messages shall be written in clear, actionable language rather than raw stack traces alone.

5.14 Compatibility

ID	Requirement
NFR-028	The mapper shall remain compatible with standard OIDC client applications that do not recognize the risk_level or restricted claims, i.e., such clients shall continue to function normally, simply ignoring the additional claims.
NFR-029	The mapper shall be compatible with Keycloak's standard ID Token and Access Token mapper attachment points, without requiring a non-standard token type.
NFR-030	Upgrading the AI Risk Assessment Service's internal scoring logic (e.g., Version 1 to Version 2 per the AI Roadmap) shall not require a corresponding change to the mapper, provided the REST contract in Section 6.4 is preserved.

6 External Interface Requirements

6.1 User Interfaces

The system defines no direct user interface for end users; end users interact only with the client application they are authenticating against, which is unaffected in appearance by this extension. Any behavioral difference (e.g., a client application choosing to restrict functionality based on the restricted claim) is the responsibility of the client application, not of this system.

6.2 Administrative Interfaces

Administrators interact with the system exclusively through Keycloak's existing Admin Console, specifically the client-level Protocol Mapper configuration screen, on which the Custom Protocol Mapper appears as a selectable mapper type once its provider JAR is deployed. No separate administrative application or console is introduced by this project.

Interface Element	Description
Mapper Type Selector	Standard Keycloak Admin Console dropdown listing available Protocol Mapper types for a client; the Custom Protocol Mapper appears here after deployment.
Configuration Form	Standard Keycloak Admin Console form rendering the configuration properties defined in Section 10, generated from the mapper's declared configuration metadata.
Realm/Client Role and Scope Management	Existing, unmodified Keycloak screens used by administrators to manage the roles and scopes that may be referenced in the mapper's suppression configuration.

6.3 Hardware Interfaces

The system defines no direct hardware interfaces. It relies only on standard server hardware capable of running the Java Virtual Machine (for Keycloak and the mapper) and a Python runtime (for the AI Risk Assessment Service), as further detailed in Section 11 (Deployment Requirements).

6.4 REST API and JSON Contracts

This section defines the interface contract between the Custom Protocol Mapper (REST client) and the AI Risk Assessment Service (REST server). The contract is intentionally minimal and versioned to preserve loose coupling, consistent with Section 5.7 (Extensibility) and Section 5.12 (Interoperability).

6.4.1 Risk Scoring Endpoint

Property	Value
Endpoint Purpose	Accepts a contextual authentication payload and returns a risk classification.
HTTP Method	POST
Path (configurable)	As configured in the AI Endpoint property (Section 10); documented default: <code>/api/v1/risk/assess</code>
Request Content Type	application/json
Response Content Type	application/json

Property	Value
Authentication	As configured in Connection Settings (Section 10); a shared-secret header is recommended for the development environment (see Section 7.2).

6.4.2 Request Payload Fields

Field	Type	Mandatory	Description
schema_version	string	Yes	Identifies the version of the context schema being used, per FR-009.
correlation_id	string	Yes	Unique identifier for tracing this request across mapper and AI service logs, per FR-014.
ip_address	string	Yes	Source IP address of the authentication event, per FR-001.
timestamp	string (ISO 8601)	Yes	Timestamp of the authentication event, per FR-002.
user_agent	string	No	User-agent string of the authenticating client, per FR-003.
last_login_timestamp	string (ISO 8601)	No	Timestamp of the user's last known login, per FR-004, where available.
realm	string	No	Name of the Keycloak realm associated with the session, provided for logging/traceability.
client_id	string	No	Identifier of the client application the user is authenticating against.

6.4.3 Response Payload Fields

Field	Type	Mandatory	Description
schema_version	string	Yes	Version of the response schema, mirroring the request's versioning approach.
correlation_id	string	Yes	Echoes the correlation identifier received in the request, per FR-014.
risk_classification	string (enum: LOW, MEDIUM, HIGH)	Yes	The assessed risk classification for the session, per FR-017.
recommended_restrictions	array of strings	No	Optional list of scope/role identifiers the AI Risk Assessment Service recommends suppressing; the mapper's configuration remains authoritative per FR-020.
evaluated_at	string (ISO 8601)	Yes	Timestamp at which the AI Risk Assessment Service completed evaluation.

6.4.4 Health Check Endpoint

Property	Value
Endpoint Purpose	Reports operational status of the AI Risk Assessment Service, per FR-046.
HTTP Method	GET
Path (documented default)	/api/v1/health
Response Fields	status (string: 'ok' or 'degraded'), version (string), uptime_seconds (number)

6.4.5 Status Codes and Error Responses

Status Code	Meaning	Mapper Behavior
200 OK	Scoring completed successfully; response body contains a valid risk_classification.	Apply FR-020 (deterministic transformation) using the returned classification.
400 Bad Request	The AI Risk Assessment Service rejected the payload as malformed or missing mandatory fields.	Treat as an invalid response; apply fallback behavior per FR-043.
401 Unauthorized / 403 Forbidden	Authentication between the mapper and the AI Risk Assessment Service failed (see Section 7.2).	Log distinctly per FR-040; apply fallback behavior per FR-043.
408 Request Timeout / connection timeout	The AI Risk Assessment Service did not respond within the configured timeout.	Apply retry logic per FR-041; if retries are exhausted, apply fallback behavior.
500 Internal Server Error	The AI Risk Assessment Service encountered an internal failure while scoring.	Treat as an invalid response; apply fallback behavior per FR-043.
503 Service Unavailable	The AI Risk Assessment Service is intentionally reporting itself as unavailable (e.g., during shutdown).	Apply fallback behavior per FR-043 without exhausting the full retry budget, if configured accordingly.

6.4.6 Required Headers

Header	Direction	Purpose
Content-Type: application/json	Request / Response	Declares the JSON payload format for both request and response bodies.
X-Correlation-Id	Request / Response	Carries the correlation identifier defined in FR-014, in addition to the payload field, for transport-level tracing (e.g., via load-balancer or gateway logs).
Authorization / X-API-Key (deployment-dependent)	Request	Carries the shared-secret or token credential used to authenticate the mapper to the AI Risk Assessment Service, per Section 7.2.

7 Security Requirements

This section defines the security posture required of the mapper and the AI Risk Assessment Service. These requirements apply in addition to, and do not relax, any of the functional and non-functional requirements defined in Sections 4 and 5.

7.1 JWT Security

- The mapper shall never alter, remove, or weaken the signature algorithm, key material, or signing process used by Keycloak to issue tokens; it shall only modify the claim set prior to signing.
- The mapper shall not introduce any claim that could be mistaken for, or override, a standard registered JWT claim (e.g., iss, sub, exp, aud) reserved for Keycloak's own use.
- Inserted claims (risk_level, restricted) shall use clearly namespaced or descriptive names to avoid collision with standard or client-specific claims.

7.2 REST Security

- Communication between the mapper and the AI Risk Assessment Service shall support authentication (e.g., a shared secret, API key, or mutual TLS) configurable per deployment, to prevent unauthorized parties from submitting scoring requests or impersonating the AI Risk Assessment Service.
- Production-like deployments shall use HTTPS for the mapper-to-AI-service connection; HTTP may be used only in isolated development/demo environments, and this constraint shall be explicitly documented.
- The AI Risk Assessment Service shall reject requests that do not present the configured authentication credential, per the status codes defined in Section 6.4.5.

7.3 Input Validation

- All fields in the context payload shall be validated for type, format, and length by the AI Risk Assessment Service before being used in scoring logic, per FR-015.
- The mapper shall validate all fields collected from the Keycloak session before transmission, per FR-006 and FR-007, to avoid forwarding malformed or unexpectedly large data.
- The AI Risk Assessment Service shall impose reasonable maximum length limits on string fields (e.g., user_agent) to mitigate resource-exhaustion risks from malformed or malicious input.

7.4 Output Validation

- The mapper shall validate that the risk_classification field returned by the AI Risk Assessment Service is one of the three defined enumerated values before applying any claim transformation, per FR-012.
- The mapper shall discard, rather than propagate, any unexpected or additional fields returned by the AI Risk Assessment Service that are not part of the documented response contract.

7.5 Logging Security

- Logs produced by the mapper and the AI Risk Assessment Service shall never contain raw user credentials, full signed tokens, or cryptographic key material, per FR-032.
- IP addresses and other contextual data included in logs shall be handled in accordance with the deploying organization's data protection obligations; this SRS does not itself mandate anonymization but requires that logging behavior be documented and configurable.

7.6 Least Privilege

- The AI Risk Assessment Service shall be granted no access whatsoever to Keycloak's database, Admin REST API, or any credential store, consistent with the Concept Note's architectural boundary.
- The mapper shall request and use only the minimum session data necessary to populate the context schema defined in Section 6.4.2.
- Any credential used to authenticate the mapper to the AI Risk Assessment Service shall be scoped solely to the scoring and health-check endpoints, and shall not grant any broader system access.

7.7 Zero Trust Principles

- No implicit trust shall be extended to a session based solely on successful credential validation; the risk-based transformation defined in Sections 4.5–4.9 is applied to every session independently, per the design principle that every login session must be evaluated independently.
- The AI Risk Assessment Service's response shall be treated as untrusted input by the mapper and validated accordingly, per FR-012, rather than assumed correct.

7.8 Secure Configuration

- Default configuration values (Section 10) shall not expose the AI Risk Assessment Service endpoint, credentials, or claim rules in a way that weakens security if left unconfigured.
- Any credential required for mapper-to-AI-service authentication shall be stored using Keycloak's standard secret-handling mechanisms (e.g., credential vaults), not in plaintext configuration where avoidable.

7.9 Timeout Handling (Security Perspective)

- Timeout values shall be bounded to a documented maximum configurable range to prevent an administrator from inadvertently configuring an unbounded wait that could be exploited as a denial-of-service vector against the login path.

7.10 Failure Behaviour (Security Perspective)

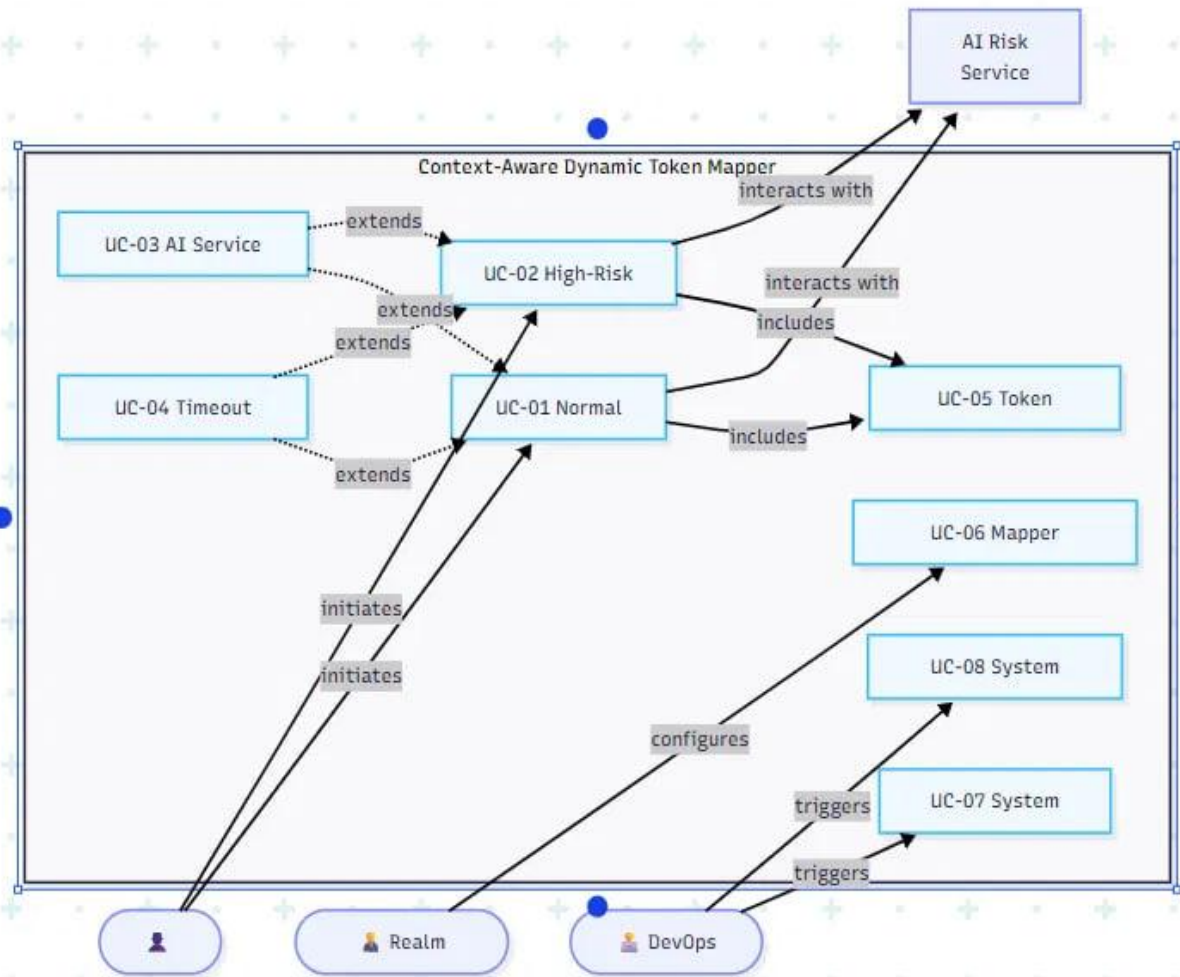
- The default (out-of-the-box) fallback posture shall be fail-closed (i.e., `restricted=true`, a conservative `risk_level`) unless an administrator explicitly reconfigures it to fail-open, consistent with secure-by-default practice.
- Failure of the AI Risk Assessment Service shall never result in privileged scopes or roles being added to a token; failure paths may only suppress privilege, never grant it.

8 System Models

This section reserves clearly labeled locations for the UML and system diagrams that accompany this SRS. Diagrams are to be produced separately (e.g., in a modeling tool) and inserted into the corresponding placeholder below; they are intentionally not reproduced as ASCII art or inline code in this document.

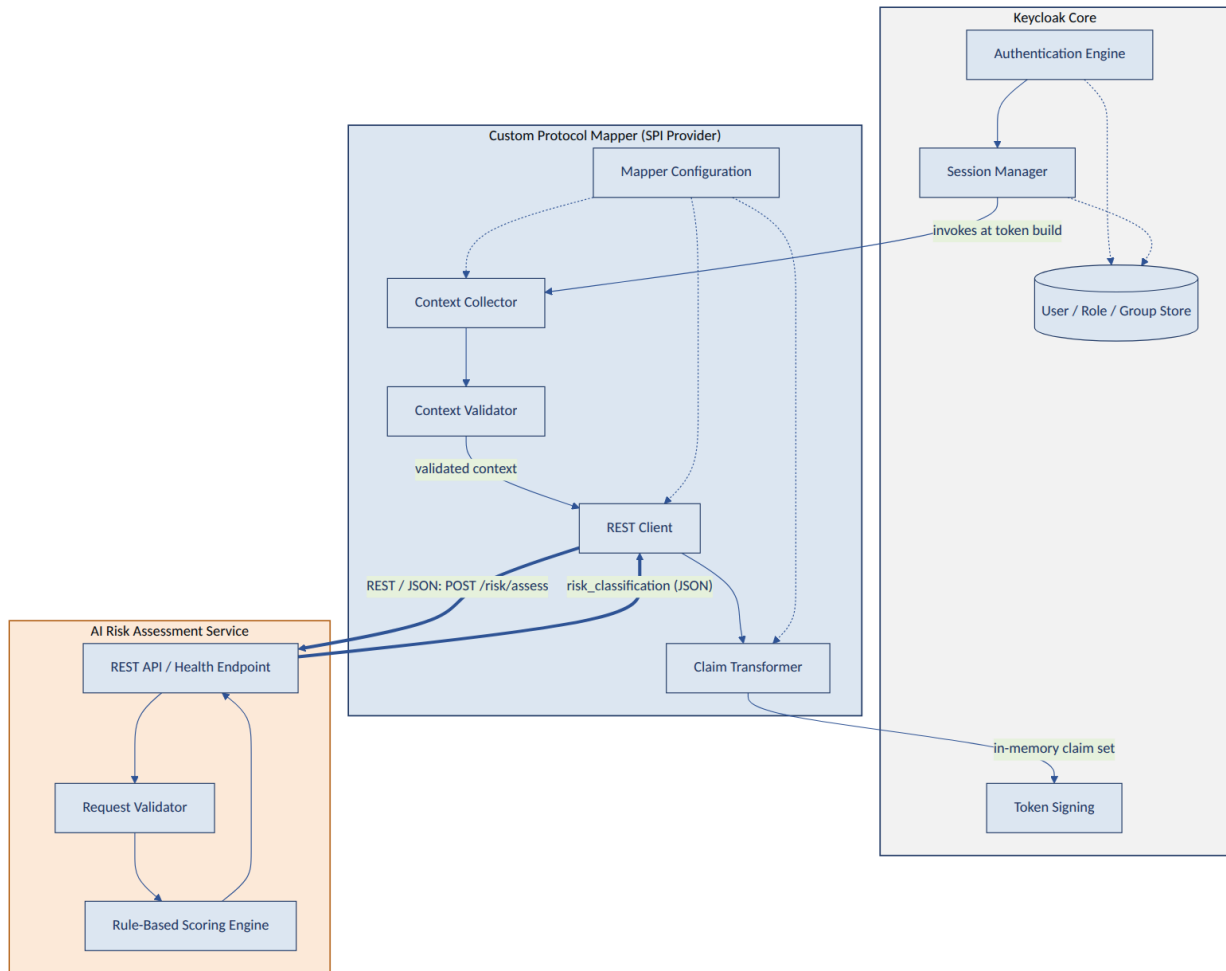
8.1 Use Case Diagram

Depicts all actors (End User, Realm Administrator, System/DevOps Engineer) and their interactions with the mapper configuration and authentication flows described in Section 9.



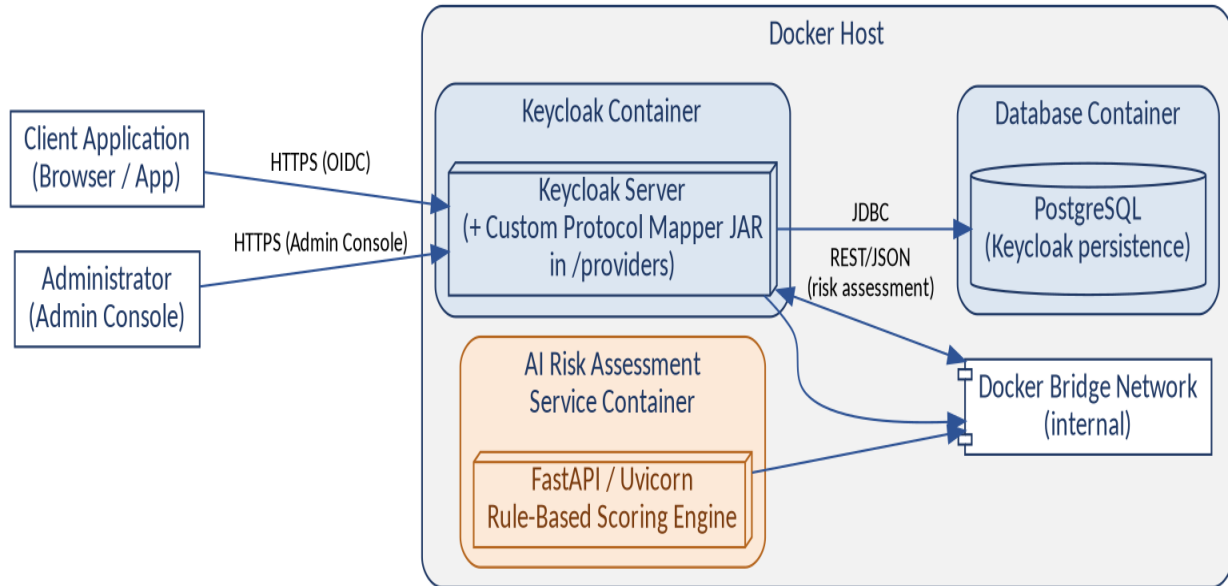
8.2 Component Diagram

Depicts the three main components (Keycloak Core, Custom Protocol Mapper, AI Risk Assessment Service) and their interfaces, consistent with Section 2.1 and the Concept Note's component table.



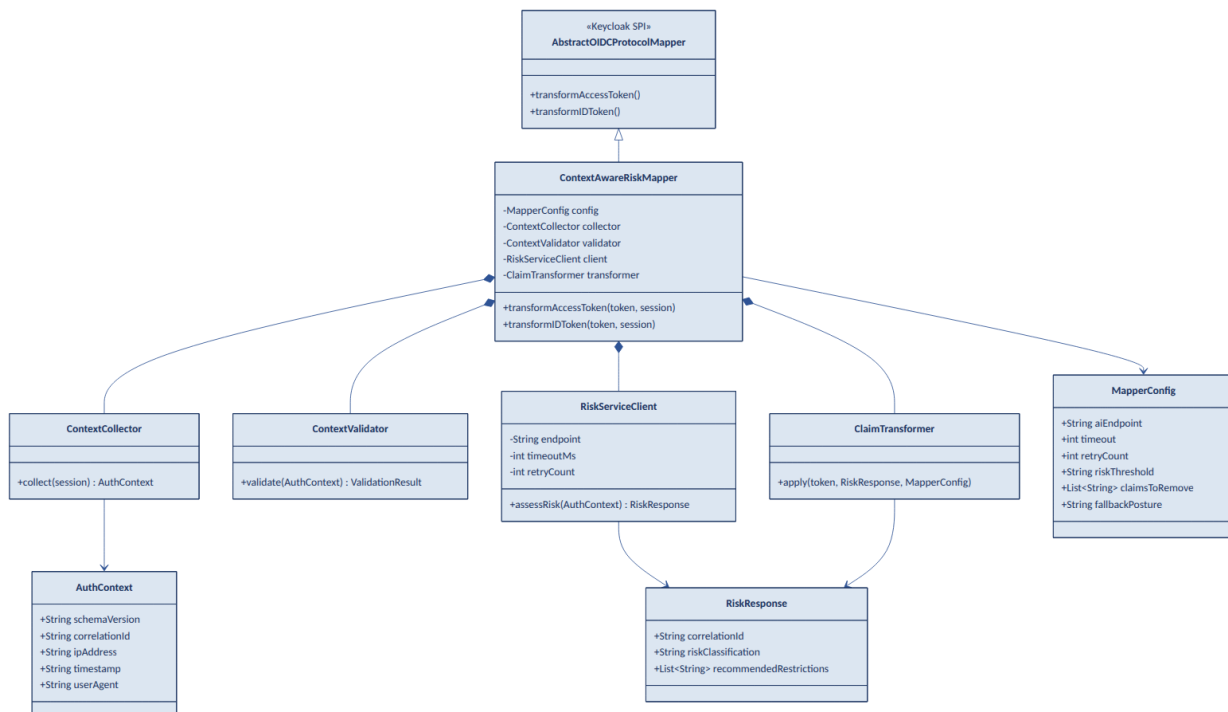
8.3 Deployment Diagram

Depicts the physical/container deployment topology described in Section 11, including the Keycloak container, database container, and AI Risk Assessment Service container.



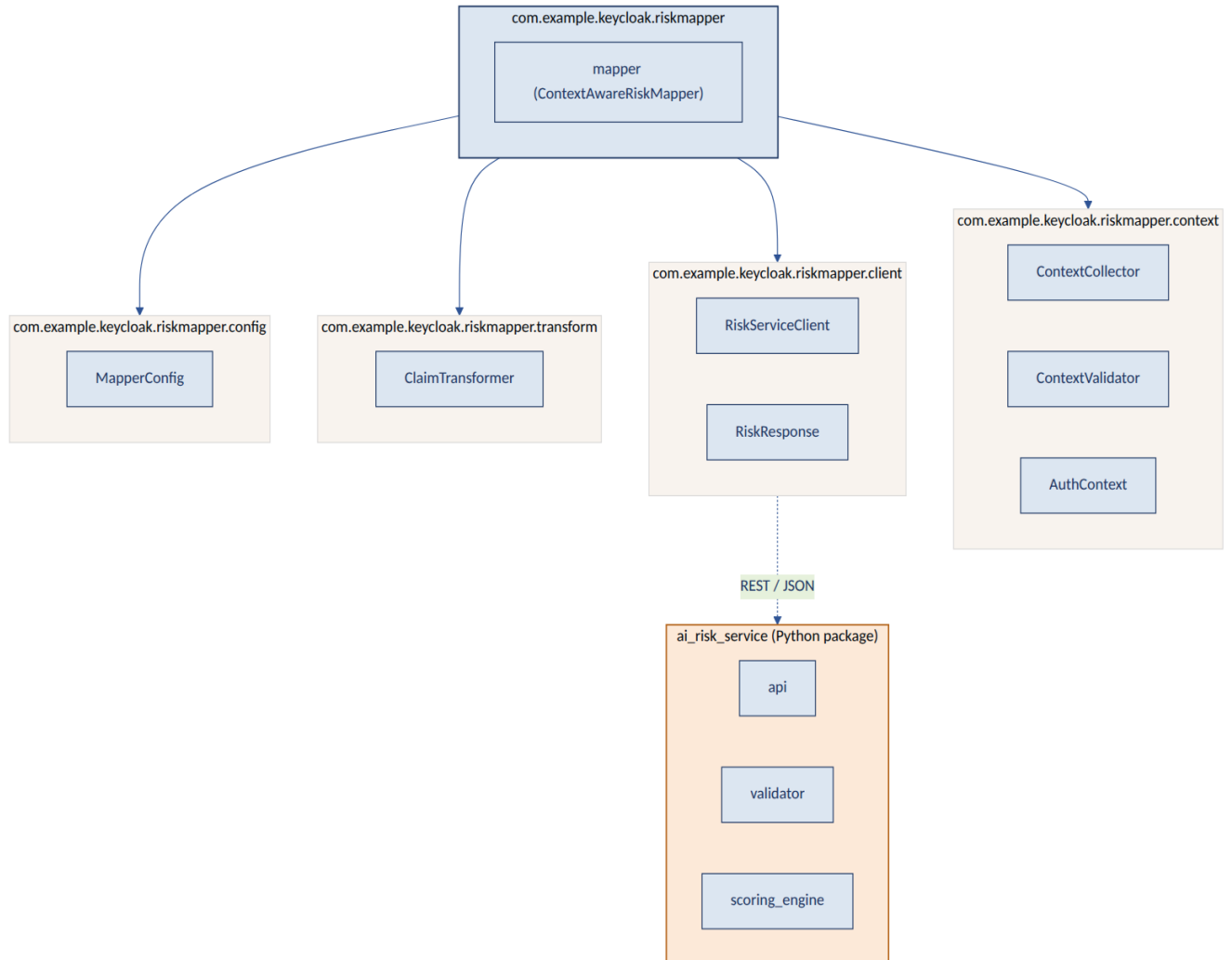
8.4 Class Diagram

Depicts the mapper's internal class structure, including its extension of AbstractOIDCProtocolMapper and its context-collection, REST-client, and claim-transformation classes.



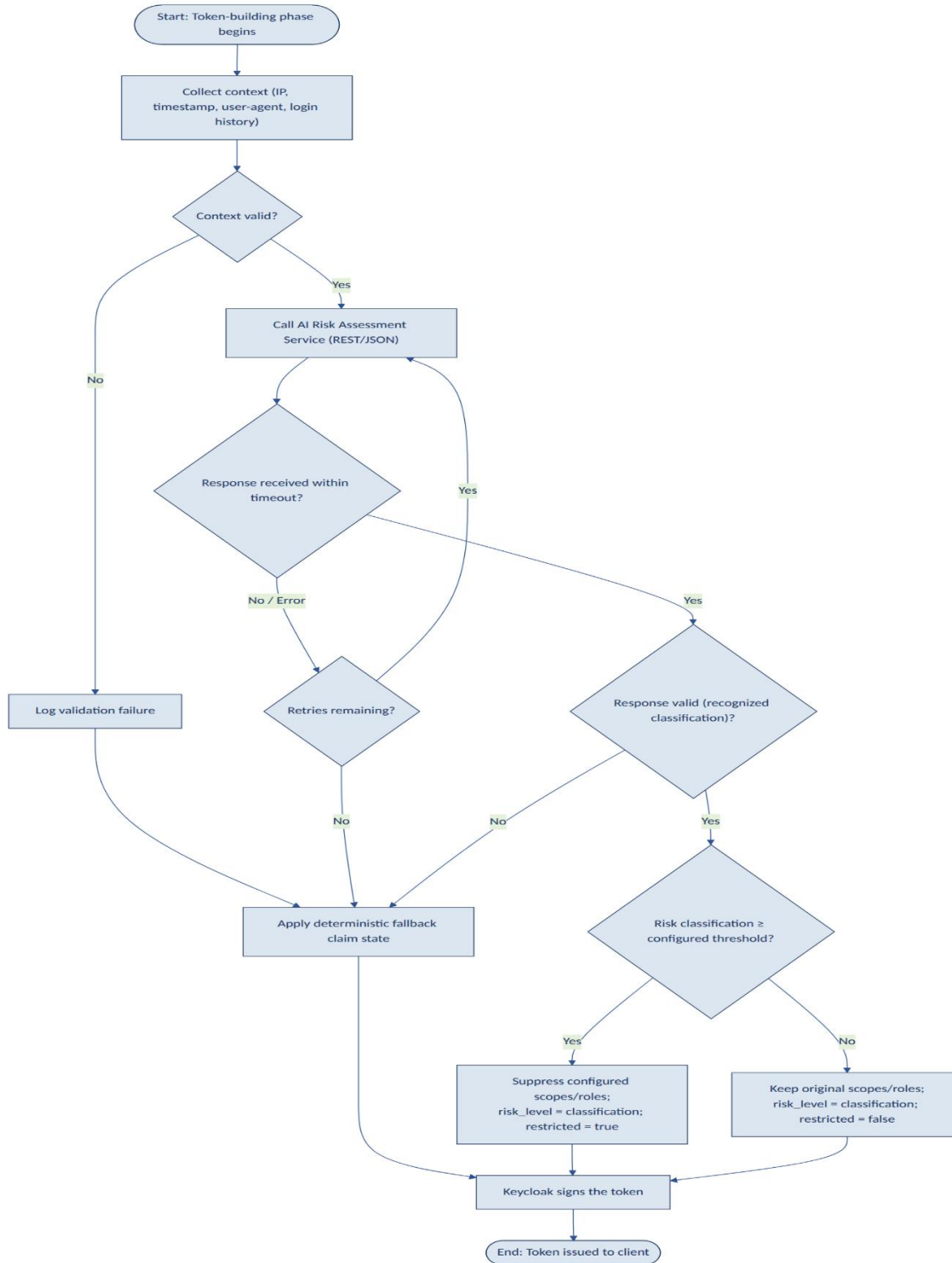
8.5 Package Diagram

Depicts the package/module organization of the mapper codebase and the AI Risk Assessment Service codebase.



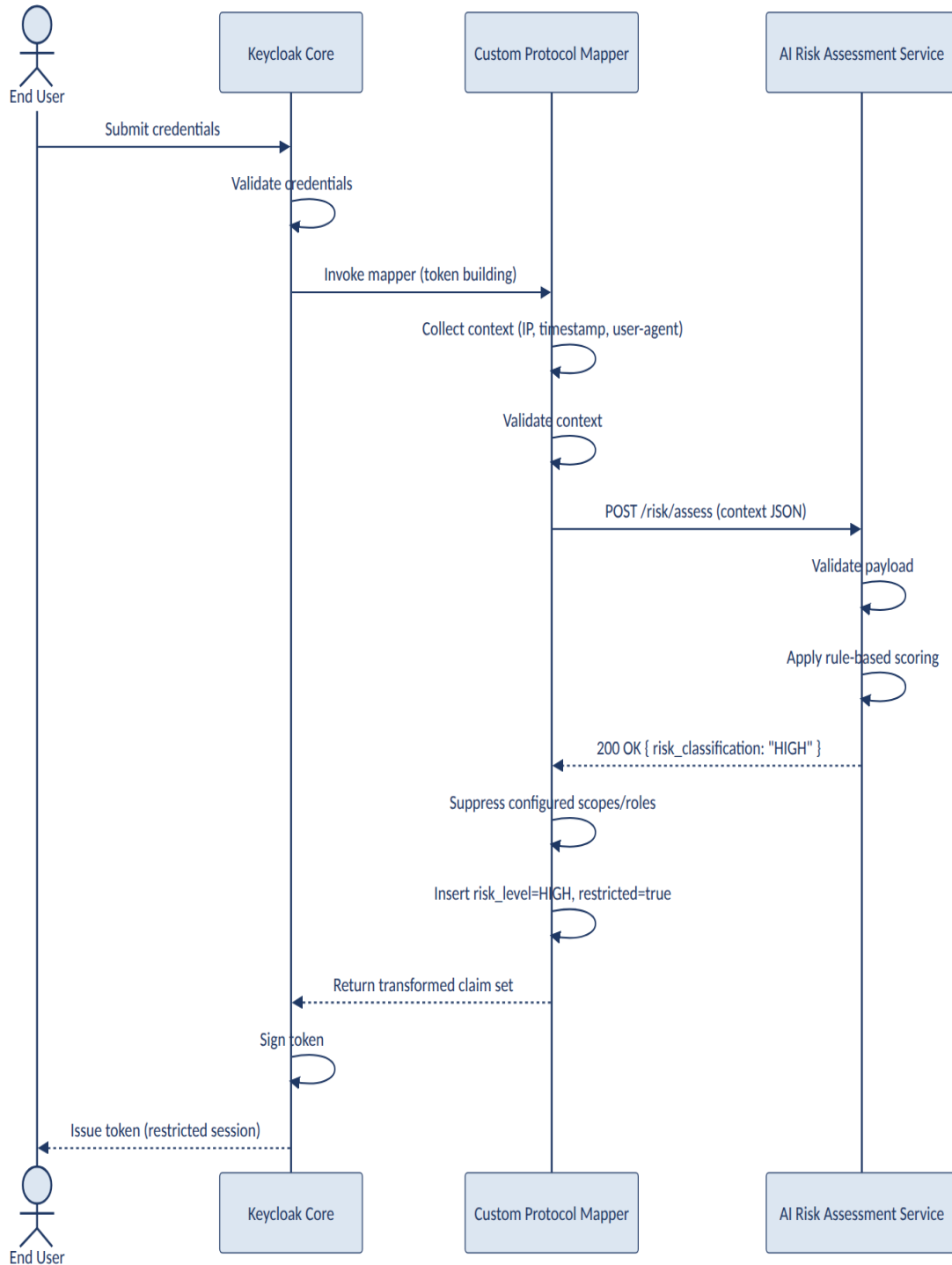
8.6 Activity Diagram

Depicts the end-to-end control flow from authentication request through claim transformation to token signing, including decision points for validation failure, timeout, and fallback.



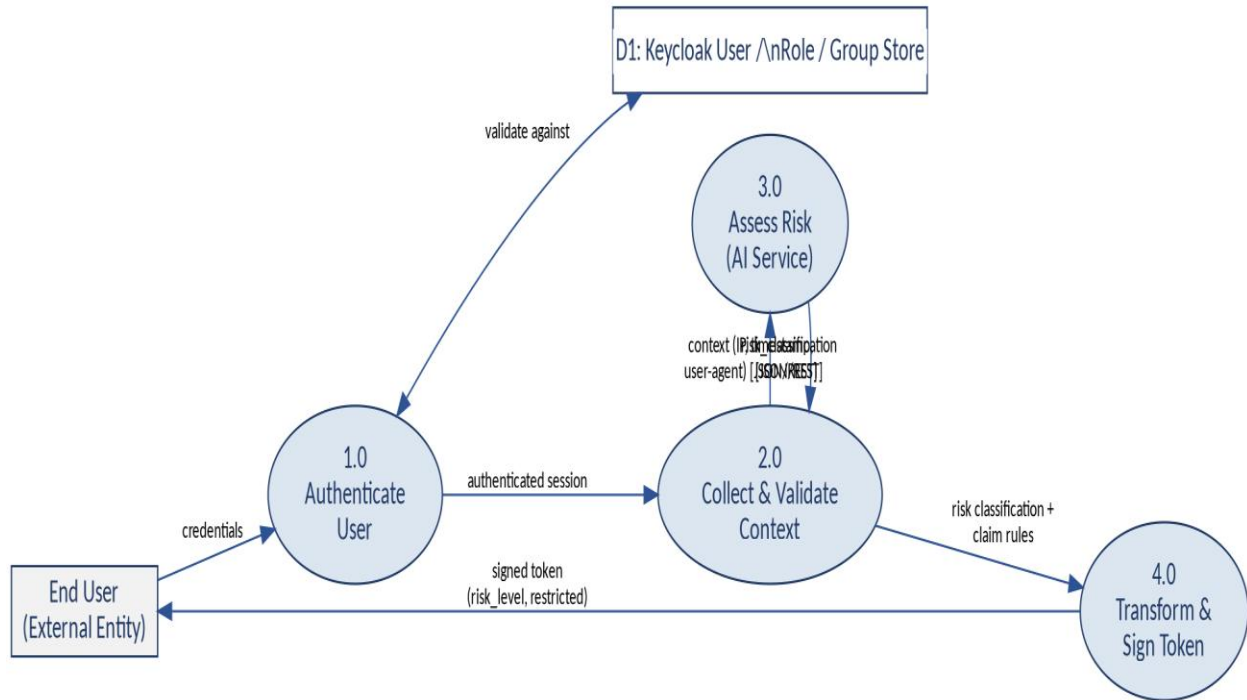
8.7 Sequence Diagram

Depicts the temporal sequence of calls between the User, Keycloak Core, the Custom Protocol Mapper, and the AI Risk Assessment Service for the Normal Login and High-Risk Login use cases in Section 9.



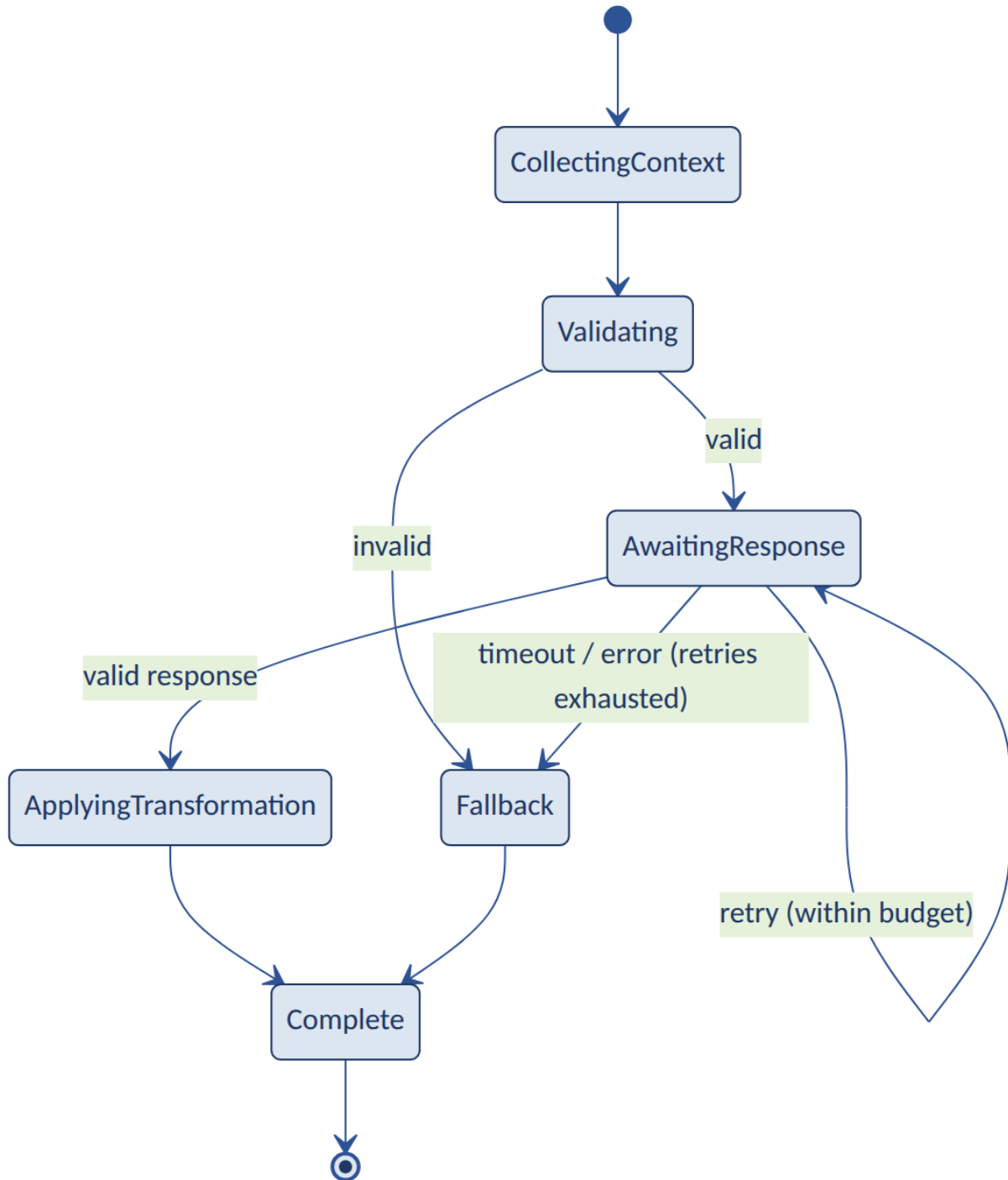
8.8 Data Flow Diagram

Depicts the flow of contextual data from session collection through validation, transmission, scoring, and claim transformation, including the fallback data path.



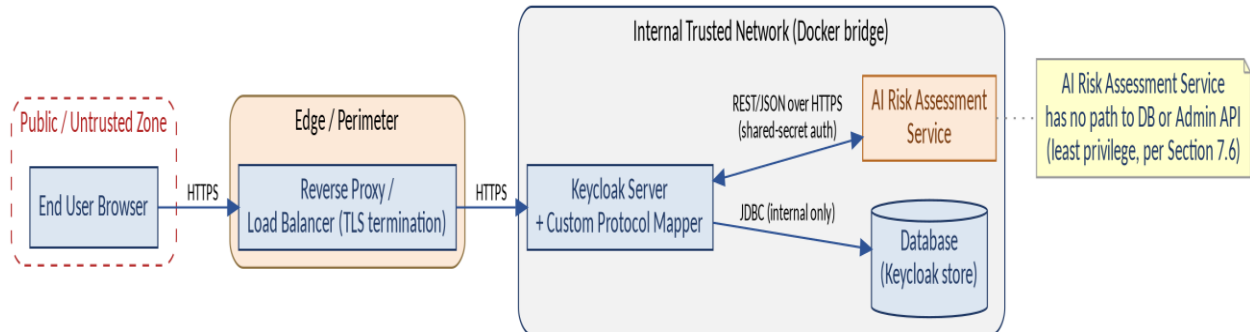
8.9 State Diagram

Depicts the states of a single mapper invocation (Collecting Context, Validating, Awaiting Response, Applying Transformation, Fallback, Complete) and the transitions between them.



8.10 Network Diagram

Depicts the network topology and trust boundaries between the Keycloak host, the AI Risk Assessment Service host, and any intermediate load balancer or reverse proxy.



9 Use Cases

Each use case below follows a consistent structure: Actors, Preconditions, Trigger, Main Flow, Alternative Flow, Exception Flow, and Postconditions, in accordance with IEEE 29148 elaboration practice for behavioral requirements.

9.1 UC-01: Normal (Low-Risk) Login

Element	Detail
Actors	End User, Keycloak Core, Custom Protocol Mapper, AI Risk Assessment Service
Preconditions	The user holds valid credentials; the client application is configured with the Custom Protocol Mapper; the AI Risk Assessment Service is running and reachable.
Trigger	The user submits valid credentials to a Keycloak-secured client application from a familiar network location and device.

Main Flow

- Keycloak validates the user's credentials and any configured authentication flows.
- Keycloak begins building the ID Token / Access Token and invokes all configured Protocol Mappers, including the Custom Protocol Mapper.
- The mapper collects contextual signals (IP address, timestamp, user-agent) per FR-001–FR-005.
- The mapper validates the context per FR-006–FR-009 and transmits it to the AI Risk Assessment Service per FR-010.
- The AI Risk Assessment Service evaluates the context and returns `risk_classification = LOW`.
- The mapper applies the configured LOW-risk transformation: no scope/role suppression; `risk_level=LOW`; `restricted=false`.
- Keycloak signs and issues the token; the user completes login with full, unrestricted privileges.

Alternative Flow

- None — this use case represents the baseline successful path.

Exception Flow

- If context validation fails unexpectedly for a normally well-formed session, the mapper logs the failure (FR-008) and proceeds via UC-03 (AI Service Failure) fallback handling rather than blocking login.

Element	Detail
Postconditions	A valid, signed token is issued containing <code>risk_level=LOW</code> and <code>restricted=false</code> ; the user's stored roles and profile are unchanged.

9.2 UC-02: High-Risk Login

Element	Detail
Actors	End User, Keycloak Core, Custom Protocol Mapper, AI Risk Assessment Service
Preconditions	The user holds valid credentials; the login originates from a context flagged as high-risk by the AI Risk Assessment Service's rule set (e.g., unfamiliar IP address and atypical time of day).
Trigger	The user submits valid credentials from a context matching one or more high-risk rules configured in the AI Risk Assessment Service.

Main Flow

- Keycloak validates the user's credentials and begins token building, invoking the Custom Protocol Mapper.
- The mapper collects and validates context, then transmits it to the AI Risk Assessment Service.
- The AI Risk Assessment Service applies its rule-based scoring function and returns `risk_classification = HIGH`.
- The mapper applies the configured HIGH-risk transformation: suppresses configured privileged scopes/roles (FR-022, FR-025); sets `risk_level=HIGH` and `restricted=true`.
- Keycloak signs and issues the token; the user completes login but with reduced effective privileges as reflected in the token claims.

Alternative Flow

- If `recommended_restrictions` are present in the AI Risk Assessment Service's response (Section 6.4.3), the mapper may use them as supplementary information, but the administrator-configured suppression list (Section 10) remains authoritative per FR-020.

Exception Flow

- If the AI Risk Assessment Service becomes unavailable partway through this flow (e.g., after the request is sent but before a response is received), the flow transitions to UC-03 (AI Service Failure).

Element	Detail
Postconditions	A valid, signed token is issued containing <code>risk_level=HIGH</code> and <code>restricted=true</code> , with configured privileged scopes/roles absent; the user's stored roles and profile in the Keycloak database remain unchanged, independently verifiable per FR-026.

9.3 UC-03: AI Risk Assessment Service Failure

Element	Detail
Actors	End User, Keycloak Core, Custom Protocol Mapper, AI Risk Assessment Service (unavailable)
Preconditions	The user holds valid credentials; the AI Risk Assessment Service is stopped, unreachable, or returning errors.
Trigger	The mapper's REST call to the AI Risk Assessment Service fails with a connection error (e.g., connection refused) rather than a timeout.

Main Flow

- Keycloak validates the user's credentials and begins token building, invoking the Custom Protocol Mapper.
- The mapper collects and validates context, then attempts the REST call to the AI Risk Assessment Service.
- The REST call fails with a connection-level error.
- The mapper retries up to the configured Retry Count (FR-041); all attempts fail.
- The mapper applies the configured fallback claim state (FR-043) and logs the fallback activation distinctly (FR-033).
- Keycloak signs and issues the token using the fallback claim state.

Alternative Flow

- If the connection error occurs on the first attempt only and a subsequent retry succeeds, the flow proceeds as UC-01 or UC-02 using the successful response instead of the fallback state.

Exception Flow

- If the fallback claim state itself cannot be applied due to a mapper-internal error (a defect condition, not an expected operating mode), the mapper shall log a critical error; this SRS requires that such a condition never suppress token issuance entirely (FR-045), and any occurrence is treated as a defect to be resolved, not as normal behavior.

Element	Detail
Postconditions	A valid, signed token is still issued, containing the documented fallback risk_level and restricted claim values; login is not blocked by AI Risk Assessment Service unavailability, per FR-045.

9.4 UC-04: Timeout Scenario

Element	Detail
Actors	End User, Keycloak Core, Custom Protocol Mapper, AI Risk Assessment Service (slow to respond)
Preconditions	The user holds valid credentials; the AI Risk Assessment Service is running but responding more slowly than the configured Timeout value.
Trigger	The AI Risk Assessment Service does not return a response within the configured Timeout window.

Main Flow

- Keycloak validates the user's credentials and begins token building, invoking the Custom Protocol Mapper.
- The mapper collects and validates context, then issues the REST call to the AI Risk Assessment Service.
- The configured Timeout elapses before a response is received (FR-039).
- The mapper logs the timeout distinctly from a connection failure (FR-040) and retries per FR-041, up to the configured Retry Count.
- If all retries also time out, the mapper applies the documented fallback claim state (FR-043).

Alternative Flow

- If a retried request completes within its own timeout window before the retry budget is exhausted, the flow proceeds using that response's risk classification instead of the fallback state.

Exception Flow

- If the underlying network stack raises an exception distinct from a simple timeout (e.g., a DNS resolution failure) during a retry attempt, the mapper treats it as a connection failure per UC-03 for logging purposes, while still counting it against the retry budget.

Element	Detail
Postconditions	A valid, signed token is issued within a bounded total time (Section 5.1, NFR-001), using either a late-but-successful response or the fallback claim state; login latency does not exceed the documented maximum bound (FR-042).

9.5 UC-05: Token Generation (Claim Transformation Detail)

Element	Detail
Actors	Keycloak Core, Custom Protocol Mapper
Preconditions	A risk classification (from the AI Risk Assessment Service or fallback) has been determined for the current session.
Trigger	Keycloak's token-building phase reaches the point of invoking the Custom Protocol Mapper's claim-transformation logic.

Main Flow

- The mapper reads the determined risk classification and the administrator's configured claim rules (Section 10).
- The mapper determines whether the risk classification meets or exceeds the configured Risk Threshold.
- If the threshold is met or exceeded, the mapper removes the configured privileged scopes/roles from the in-memory claim set (FR-022, FR-025).
- The mapper inserts the risk_level claim with the determined classification value (FR-027).
- The mapper inserts the restricted claim with a boolean value reflecting the threshold comparison (FR-029).
- Control returns to Keycloak, which proceeds to sign the token using the finalized claim set.

Alternative Flow

- If the administrator has configured no privileged scopes/roles for suppression, only the risk_level and restricted claims are inserted, with no scope/role change.

Exception Flow

- If claim insertion fails due to a naming collision with an existing, non-overridable standard claim, the mapper logs an error and issues the token without the colliding custom claim, rather than failing token issuance.

Element	Detail
Postconditions	The token signed by Keycloak reflects exactly the claim set determined by this use case; no persisted data is touched at any point in this flow.

9.6 UC-06: Mapper Configuration by an Administrator

Element	Detail
Actors	Keycloak Realm Administrator
Preconditions	The administrator has access to the Keycloak Admin Console with sufficient privileges to manage client Protocol Mappers; the mapper provider JAR has been deployed (see UC-07).
Trigger	The administrator navigates to a client's Mappers tab and chooses to add or edit a Custom Protocol Mapper instance.

Main Flow

- The administrator selects the Custom Protocol Mapper type from the mapper type list (FR-035, FR-048).
- The Admin Console renders the configuration form with fields for every property in Section 10.
- The administrator enters or updates values (e.g., AI Endpoint, Timeout, Risk Threshold, Claims To Remove).
- The administrator saves the configuration; the Admin Console validates the entered values (FR-036).
- The updated configuration takes effect on the next authentication request for that client, without a Keycloak restart (FR-037).

Alternative Flow

- The administrator may instead accept the documented default values (FR-038) and save without customization, resulting in safe, documented default behavior.

Exception Flow

- If the administrator enters an invalid value (e.g., a non-numeric Timeout), the Admin Console rejects the save with a descriptive validation message and the previous configuration remains in effect.

Element	Detail
Postconditions	The client's Custom Protocol Mapper configuration is persisted in Keycloak's standard configuration store and is applied to subsequent logins for that client.

9.7 UC-07: System Startup

Element	Detail
Actors	System/DevOps Engineer, Keycloak Core, AI Risk Assessment Service
Preconditions	The mapper provider JAR has been placed in Keycloak's providers directory; the Docker Compose configuration (or equivalent manual deployment) is available.
Trigger	The engineer starts the environment (e.g., via docker compose up, or by starting Keycloak and the AI Risk Assessment Service individually).

Main Flow

- The AI Risk Assessment Service starts and begins listening on its configured port, exposing the health endpoint (FR-046).
- Keycloak starts and discovers the Custom Protocol Mapper provider JAR during its provider build/registration step (FR-048).
- If the optional connectivity check (FR-047) is enabled, the mapper performs a non-blocking check of the configured AI endpoint and logs the result without delaying Keycloak startup.
- Keycloak completes startup and becomes ready to accept authentication requests, with the Custom Protocol Mapper available for client configuration.

Alternative Flow

- If the AI Risk Assessment Service has not yet finished starting when Keycloak completes startup, this is not treated as an error; the first authentication attempt will invoke the standard timeout/retry/fallback path (UC-03/UC-04) until the AI Risk Assessment Service becomes reachable.

Exception Flow

- If the mapper provider JAR is malformed or fails Keycloak's provider registration, Keycloak logs a provider registration error consistent with its standard SPI error handling; this SRS requires that such a failure not prevent Keycloak from starting to serve realms/clients that do not reference the mapper.

Element	Detail
Postconditions	Both Keycloak (with the Custom Protocol Mapper registered) and the AI Risk Assessment Service (with its health endpoint reporting 'ok') are running and ready to process authentication requests.

9.8 UC-08: System Shutdown

Element	Detail
Actors	System/DevOps Engineer, Keycloak Core, AI Risk Assessment Service
Preconditions	The environment is running as described in UC-07.
Trigger	The engineer stops the environment (e.g., via docker compose down, or by stopping the individual services).

Main Flow

- The engineer initiates shutdown of the AI Risk Assessment Service, the Keycloak server, or both.
- In-flight authentication requests that had already received a risk classification complete token issuance normally before shutdown takes effect.
- Any authentication request that begins after the AI Risk Assessment Service has stopped, but while Keycloak remains running, follows the fallback path described in UC-03.
- Once both components are stopped, no further authentication requests can be processed until the system is restarted (UC-07).

Alternative Flow

- The AI Risk Assessment Service may be stopped independently of Keycloak (e.g., for maintenance) without requiring Keycloak to also be stopped, consistent with the components' loose coupling.

Exception Flow

- If Keycloak is stopped abruptly (e.g., process killed) mid-request, standard Keycloak session/transaction handling applies; this SRS does not alter Keycloak's own shutdown behavior, per the constraint that core Keycloak logic is not modified.

Element	Detail
Postconditions	Both components reach a stopped state with no residual effect on persisted Keycloak data; the system can be restarted per UC-07 without data migration or cleanup.

10 Configuration Requirements

This section documents every administrator-configurable property of the Custom Protocol Mapper, consistent with FR-035 through FR-038. All properties are exposed through the Keycloak Admin Console's Protocol Mapper configuration screen for the client to which the mapper is attached.

Property	Description	Type	Default	Example
AI Endpoint	The base URL of the AI Risk Assessment Service's scoring endpoint (Section 6.4.1).	String (URL)	http://ai-risk-service:8000/api/v1/risk/assess	https://risk.internal.example.com/api/v1/risk/assess
Timeout	Maximum time, in milliseconds, the mapper waits for a response before treating the call as failed (FR-039).	Integer (ms)	500	800
Retry Count	Maximum number of retry attempts after a timeout or connection failure before invoking fallback (FR-041).	Integer	1	2
Risk Threshold	The minimum risk classification (MEDIUM or HIGH) at which scope/role suppression and restricted=true are applied (FR-029).	Enum (MEDIUM, HIGH)	HIGH	MEDIUM
Logging Level	Verbosity of mapper log output (FR-031).	Enum (ERROR, WARN, INFO, DEBUG)	INFO	DEBUG

Property	Description	Type	Default	Example
Claims To Remove	Comma-separated list of scope and/or role identifiers to suppress when the threshold is met (FR-022, FR-025).	String (list)	(empty)	admin:write, realm-admin
Claims To Add	Additional static claim name/value pairs to insert alongside risk_level and restricted, if any (extensibility hook per NFR-014).	String (key=value list)	(empty)	policy_version=1
Cache Duration	Optional duration, in seconds, for which an identical context's risk classification may be reused without a new AI Risk Assessment Service call, to reduce latency for rapid repeated logins.	Integer (seconds)	0 (disabled)	30
Connection Settings	Authentication credential (e.g., API key) and TLS verification behavior for the mapper-to-AI-service connection (Section 7.2).	Structured (key/value)	TLS verification enabled; no credential	api_key=<secret>; verify_tls=true
Risk Claim Name	Configurable name of the inserted risk claim (FR-028).	String	risk_level	auth_risk

Property	Description	Type	Default	Example
Restricted Claim Name	Configurable name of the inserted restricted claim (FR-030).	String	restricted	reduced_privilege
Fallback Posture	Determines whether fallback applies a conservative (fail-closed) or permissive (fail-open) claim state (FR-044).	Enum (FAIL_CLOSED, FAIL_OPEN)	FAIL_CLOSED	FAIL_OPEN

11 Deployment Requirements

11.1 Development Environment

Tool / Component	Role
Docker	Container runtime hosting Keycloak, its database, and the AI Risk Assessment Service.
Docker Compose	Orchestrates the multi-container local development and demonstration environment (FR-049).
Keycloak (open-source)	Target IAM platform; hosts the deployed Custom Protocol Mapper provider JAR.
Java Development Kit	Builds and runs the mapper, matching Keycloak's supported JVM version.
Apache Maven	Builds, tests, and packages the mapper as a provider JAR (FR-048).
Python 3	Runtime for the AI Risk Assessment Service.
FastAPI / Uvicorn	Web framework and ASGI server hosting the AI Risk Assessment Service's REST endpoints.
Git / GitHub	Version control and source hosting for both the mapper and the AI Risk Assessment Service codebases.
Postman (or equivalent HTTP client)	Manual and scripted testing of the AI Risk Assessment Service's REST endpoints (Section 12.2).
JWT Decoder (e.g., jwt.io or CLI equivalent)	Manual inspection of issued tokens during testing to confirm claim transformations.

11.2 Operating Systems

The development and demonstration environment shall be runnable on any host operating system supporting Docker and Docker Compose, including common Linux distributions, macOS, and Windows with WSL2. No component of the system shall depend on operating-system-specific behavior, consistent with the Portability requirements in Section 5.11.

11.3 Build Process

- The mapper shall be built using a single Maven command (e.g., `mvn clean package`) that produces a deployable provider JAR without manual post-processing.
- The AI Risk Assessment Service shall be built/packaged using a standard Python dependency mechanism (e.g., a requirements file or equivalent), sufficient for containerization.
- Both build processes shall be repeatable and shall not depend on developer-machine-specific state.

11.4 Packaging

- The mapper shall be packaged as a single JAR file containing the compiled mapper classes and the SPI provider registration metadata required by Keycloak.
- The AI Risk Assessment Service shall be packaged as a container image built from a documented Dockerfile, exposing its REST port.

11.5 Deployment Steps

Step	Action
1	Build the mapper provider JAR using Maven.
2	Place the built JAR into Keycloak's providers directory.
3	Run Keycloak's provider build step so the new SPI provider is registered.
4	Build and start the AI Risk Assessment Service container (or start it via the provided Docker Compose file).
5	Start Keycloak (or start the full environment via Docker Compose).
6	In the Admin Console, attach the Custom Protocol Mapper to the desired client and configure it per Section 10.
7	Perform a test login and inspect the issued token to confirm expected claim behavior, per Section 12.

12 Testing Requirements

Testing shall be organized into the following categories, each targeting a distinct aspect of correctness, resilience, and security. At least 30 realistic test cases are defined below, each with a unique identifier, objective, preconditions, inputs, expected result, and acceptance criteria.

12.1 Testing Categories

Category	Objective
Unit Testing	Verify individual classes/functions (context collection, validation, claim transformation, scoring rules) in isolation, without a running Keycloak server or live AI Risk Assessment Service.
Integration Testing	Verify correct interaction between the mapper and a running AI Risk Assessment Service instance over REST.
System Testing	Verify correct end-to-end behavior of the full stack (Keycloak + mapper + AI Risk Assessment Service) for realistic login scenarios.
Security Testing	Verify that security requirements in Section 7 are enforced (input validation, authentication between components, absence of sensitive data in logs).
Performance Testing	Verify that latency requirements in Section 5.1 (NFR-001, NFR-002) are met under normal and degraded conditions.
Regression Testing	Re-verify previously passing test cases after any change to the mapper, the AI Risk Assessment Service, or their configuration.
Acceptance Testing	Verify that the system satisfies the specific objectives of the Concept Note (Section 5.2 of the Concept Note), particularly the non-destructive, session-scoped nature of the design.

12.2 Test Cases

Test ID	Objective	Preconditions	Inputs	Expected Result	Acceptance Criteria
TC-U-01	Context object assembles all available fields	Mocked Keycloak session with IP, timestamp, user-agent available	Mocked session object	Context object contains ip_address, timestamp, user_agent	All three fields present and correctly mapped in unit test assertion
TC-U-02	Context collection omits unavailable login history	Mocked session with no login-history session note	Mocked session object without history	last_login_timestamp field is omitted, not null-filled	Unit test asserts field absence rather than a placeholder value
TC-U-03	Validation rejects missing timestamp	Context object missing the timestamp field	Context object without timestamp	Validation function returns invalid, with reason 'timestamp missing'	Unit test asserts rejection and correct reason code
TC-U-04	Validation rejects malformed IP address	Context object with ip_address = 'not-an-ip'	Malformed IP string	Validation function returns invalid	Unit test asserts rejection for malformed IP
TC-U-05	Claim transformation	Mocked risk classification =	Risk classification,	Resulting claim set omits 'admin:write'	Unit test asserts scope absent from

Test ID	Objective	Preconditions	Inputs	Expected Result	Acceptance Criteria
	suppresses configured scopes for HIGH risk	HIGH; Claims To Remove = ['admin:write']	mapper configuration		transformed claim set
TC-U-06	Claim transformation leaves claims unchanged for LOW risk	Mocked risk classification = LOW	Risk classification, mapper configuration	Resulting claim set unchanged except risk_level/restricted claims added	Unit test asserts all pre-existing scopes/roles preserved
TC-U-07	risk_level claim uses configured custom name	Risk Claim Name configured as 'auth_risk'	Mapper configuration, risk classification = MEDIUM	Claim named 'auth_risk' with value 'MEDIUM' inserted	Unit test asserts custom claim name and value
TC-U-08	Rule-based scoring function is deterministic	Fixed context payload, fixed rule configuration	Identical context payload submitted twice	Identical risk_classification returned both times	Unit test asserts equality of two scoring calls
TC-I-01	Mapper successfully calls a running AI Risk Assessment Service	AI Risk Assessment Service running and reachable	Valid context payload from a test login	HTTP 200 response with valid risk_classification	Integration test confirms one REST call and a parsed, valid response
TC-I-02	Mapper handles a 400 Bad Request response	AI Risk Assessment Service configured to reject a specific test payload	Deliberately malformed context payload	Mapper treats response as invalid and applies fallback	Integration test confirms fallback claim state applied after 400 response
TC-I-03	Mapper authenticates to the AI Risk Assessment Service	Connection Settings configured with a valid API key	Standard context payload	Request includes configured credential header; AI service accepts it	Integration test confirms 200 response only when credential is present and correct
TC-I-04	Mapper rejected when credential is missing or incorrect	Connection Settings configured with an invalid API key	Standard context payload	AI Risk Assessment Service returns 401/403; mapper applies fallback	Integration test confirms fallback triggered on authentication failure
TC-I-05	Correlation ID matches across mapper and AI service logs	Both components configured with DEBUG logging	Standard test login	Same correlation_id appears in both logs	Log inspection confirms matching correlation identifiers for the same request
TC-I-06	Health endpoint reports 'ok' when service is running normally	AI Risk Assessment Service running normally	GET request to health endpoint	HTTP 200 with status='ok'	Integration test confirms expected health response body
TC-S-01	End-to-end LOW-risk login issues unrestricted token	Full stack running; test user configured with	Login request from familiar IP/device	Issued token: risk_level=LOW, restricted=false, all	Decoded token matches expected claim set exactly

Test ID	Objective	Preconditions	Inputs	Expected Result	Acceptance Criteria
		low-risk context (known IP)		original scopes/roles present	
TC-S-02	End-to-end HIGH-risk login issues restricted token	Full stack running; test context matches a HIGH-risk rule (unfamiliar IP, atypical time)	Login request from unfamiliar IP	Issued token: risk_level=HIGH, restricted=true, configured scopes/roles suppressed	Decoded token matches expected suppressed claim set exactly
TC-S-03	End-to-end MEDIUM-risk login applies partial restriction	Full stack running; test context matches a MEDIUM-risk rule	Login request with moderately unusual context	Issued token: risk_level=MEDIUM; restriction applied only if threshold configured to MEDIUM	Decoded token claim set matches configuration for MEDIUM threshold setting
TC-S-04	Stored user roles unchanged after HIGH-risk login	Test user with known role assignments; full stack running	HIGH-risk login followed by Admin Console role inspection	User's stored roles identical before and after login	Admin Console (or Admin REST API) shows zero difference in assigned roles
TC-S-05	Mapper can be disabled without residual effect	Custom Protocol Mapper removed from test client configuration	Standard login request	Issued token contains standard claims only, no risk_level/restricted claims	Decoded token shows no custom claims and no errors logged
TC-S-06	Configuration change takes effect without restart	Full stack running; Risk Threshold changed from HIGH to MEDIUM via Admin Console	MEDIUM-risk login performed immediately after the change	Suppression applied at MEDIUM threshold without a Keycloak restart	Decoded token reflects new threshold behavior on the very next login
TC-S-07	Multiple concurrent sessions are evaluated independently	Two simultaneous logins from different users with different contexts	Concurrent login requests	Each session receives an independently correct risk classification and claim set	Both decoded tokens match their respective expected outcomes with no cross-session interference
TC-SEC-01	Logs do not contain raw user passwords	DEBUG logging enabled; standard login performed	Standard login request with known password	Password string does not appear anywhere in mapper or AI service logs	Text search of log files for the literal password string returns zero matches
TC-SEC-02	Logs do not contain the full signed JWT	DEBUG logging enabled; standard login performed	Standard login request	Full signed token string does not appear in logs (summary/claims only)	Text search of log files for the full token string returns zero matches

Test ID	Objective	Preconditions	Inputs	Expected Result	Acceptance Criteria
TC-SEC-03	AI Risk Assessment Service rejects requests without valid credential	Connection Settings require an API key; service running	Request sent with missing/invalid API key	HTTP 401/403 returned; no risk classification computed	Response status code confirms rejection prior to scoring logic execution
TC-SEC-04	AI Risk Assessment Service rejects oversized user_agent field	Maximum length limit configured for user_agent field	Context payload with an excessively long user_agent string	HTTP 400 returned; request rejected before scoring	Response confirms rejection with a length-related validation error
TC-SEC-05	TLS is used when AI Endpoint is configured with https://	AI Endpoint configured with an https:// URL and valid certificate	Standard login request	Outbound request uses TLS; no plaintext HTTP observed on the wire	Network capture confirms TLS handshake and encrypted payload
TC-SEC-06	Fallback never grants additional privilege	AI Risk Assessment Service stopped; Fallback Posture = FAIL_CLOSED	Standard login request	Issued token has restricted=true and no privileged scopes added	Decoded token contains no privileged scope/role not already held by the user before login
TC-P-01	Mapper adds acceptable latency under normal AI response time	AI Risk Assessment Service responding within 100ms	100 sequential login requests	95th percentile added latency $\leq$ 1.5 seconds (NFR-001)	Measured 95th percentile latency meets or is below the NFR-001 bound
TC-P-02	AI Risk Assessment Service responds within target latency	AI Risk Assessment Service under normal load	100 sequential scoring requests	95th percentile response time $\leq$ 300ms (NFR-002)	Measured 95th percentile response time meets or is below the NFR-002 bound
TC-P-03	Total worst-case latency stays within documented bound	Timeout=500ms, Retry Count=2, AI Risk Assessment Service stopped	Single login request	Total added latency $\approx$ $\leq$ 1.5 seconds before fallback is applied (FR-042)	Measured total time from request to fallback claim application does not exceed the documented bound
TC-R-01	Previously passing LOW-risk scenario still passes after a configuration change	TC-S-01 previously passed; unrelated configuration property changed	Re-run of TC-S-01 inputs	Identical result to original TC-S-01 execution	No deviation from the previously recorded expected result
TC-R-02	Previously passing fallback scenario still passes after an AI Risk Assessment Service upgrade	TC-S-05/TC-SEC-06 previously passed; AI Risk Assessment Service version upgraded per	Re-run of the fallback test inputs	Identical fallback behavior observed, per NFR-030 (mapper decoupling from AI internals)	No change in mapper-observable behavior despite the AI Risk Assessment Service internal upgrade

Test ID	Objective	Preconditions	Inputs	Expected Result	Acceptance Criteria
		Section 14 roadmap			
TC-A-01	System satisfies non-destructive, session-scoped design objective	Full stack running; test user with known role/attribute baseline	A sequence of LOW, MEDIUM, and HIGH risk logins for the same user	Stored user data identical throughout; only issued tokens vary	Database/Admin Console inspection after all three logins shows zero change to stored roles, groups, or attributes
TC-A-02	System satisfies the resilient-integration objective	Full stack running; AI Risk Assessment Service stopped mid-test	A login attempt during AI Risk Assessment Service downtime	Login completes successfully via fallback; no user-facing authentication failure	End-to-end login succeeds and is indistinguishable in outcome (aside from claim values) from a normal login
TC-A-03	System satisfies the configurability objective	Full stack running	A full pass through Admin Console configuration of every property in Section 10	Every property in Section 10 is visible, editable, and takes effect as documented	Manual walkthrough confirms all configuration properties behave as specified without needing source code access

A total of 35 test cases are defined above, spanning Unit, Integration, System, Security, Performance, Regression, and Acceptance categories, satisfying the requirement for at least 30 realistic test cases.

13 Risks

This section documents risks identified for the project across five categories. Each risk includes a description, a qualitative probability and impact rating, and a mitigation strategy.

13.1 Technical Risks

Risk	Description	Probability	Impact	Mitigation
T-01	Keycloak's SPI/session APIs do not expose a needed contextual attribute (e.g., detailed login history) in the target version.	Medium	Medium	Scope context collection to attributes confirmed available in the target Keycloak version during design (FR-004 already treats login history as best-effort).
T-02	The mapper introduces a defect that inadvertently modifies persisted user data.	Low	High	Enforce FR-019/FR-026 through automated tests (TC-A-01) run on every build; code review focused specifically on any write path to Keycloak's persistence layer.
T-03	Incompatibility between the mapper's build toolchain and a future Keycloak version.	Medium	Medium	Pin the Keycloak/SPI version used for development and document the supported version range explicitly in deployment documentation.

13.2 Operational Risks

Risk	Description	Probability	Impact	Mitigation
O-01	Administrators misconfigure the AI Endpoint or credentials, causing persistent fallback activation.	Medium	Low	Provide clear configuration documentation (FR-050), sensible defaults (FR-038), and a connectivity check on startup (FR-047).
O-02	Logging is left at DEBUG level in an environment where log volume becomes operationally burdensome.	Low	Low	Document a recommended production logging level and make Logging Level configurable without restart (FR-037).
O-03	The Docker Compose demonstration environment drifts from the documented deployment steps over time.	Low	Low	Version-control the Docker Compose file alongside the codebase and re-validate it as part of regression testing (Section 12.1).

13.3 Security Risks

Risk	Description	Probability	Impact	Mitigation
S-01	The AI Risk Assessment Service is reachable without authentication in a misconfigured deployment.	Medium	High	Default configuration documentation strongly recommends enabling Connection Settings authentication; TC-SEC-03 verifies enforcement when configured.
S-02	Sensitive data (e.g., IP addresses) transmitted to the AI Risk Assessment Service is exposed due to lack of TLS in a production-like deployment.	Medium	Medium	Document HTTPS as the required transport for non-development deployments (Section 7.2); verify via TC-SEC-05.
S-03	A malicious or malformed AI Risk Assessment Service response is used to escalate privilege.	Low	High	Strict output validation (FR-012, Section 7.4) ensures only recognized, enumerated risk classifications are acted upon; fallback never grants additional privilege (Section 7.10).

13.4 Performance Risks

Risk	Description	Probability	Impact	Mitigation
P-01	AI Risk Assessment Service latency under load exceeds the target response time, degrading login responsiveness.	Medium	Medium	Enforce NFR-002 through performance testing (TC-P-02); timeout and retry bounds (FR-039, FR-042) cap the worst-case impact on login latency.
P-02	Retry logic under repeated failures adds unacceptable cumulative latency to the login path.	Low	Medium	Bound total retry duration explicitly (FR-042) and verify via TC-P-03.

13.5 Integration Risks

Risk	Description	Probability	Impact	Mitigation
I-01	A future change to the AI Risk Assessment Service's response schema breaks compatibility with the deployed mapper.	Medium	Medium	Enforce schema versioning on both request and response payloads (FR-009, Section 6.4.3) and require the mapper to reject/fallback on unrecognized schema versions.
I-02	Network segmentation between the Keycloak host and the AI Risk Assessment Service host is not established before testing, causing all logins to hit fallback unexpectedly.	Low	Low	Validate network reachability as part of the deployment steps (Section 11.5, Step 4–5) before beginning functional testing.

14 Future Enhancements

This section documents the planned evolution of the AI Risk Assessment Service beyond Version 1, consistent with the AI Roadmap defined in the Concept Note. None of the enhancements below require changes to the mapper's REST contract (Section 6.4) or to Keycloak's core authentication logic, because the AI Service is already modular and loosely coupled to the mapper through a versioned JSON interface (NFR-015, NFR-030).

14.1 Version 2 — Machine Learning Risk Engine

Version 2 replaces the Version 1 rule-based scoring function with a trained machine learning model, while preserving the exact request/response contract defined in Section 6.4. Candidate algorithms identified in the Concept Note include:

- Decision Tree — a simple, interpretable classifier suitable as a first machine learning iteration.
- Random Forest — an ensemble method offering improved accuracy over a single decision tree.
- Logistic Regression — a lightweight probabilistic classifier well suited to a binary or ordinal risk output.

Because the mapper only ever consumes a validated `risk_classification` value over REST, migrating from a rule-based engine to any of the above algorithms requires no change to the mapper, Keycloak, or the token-issuance pipeline — only to the internal scoring logic of the AI Risk Assessment Service.

14.2 Version 3 — Advanced AI

Version 3 introduces more sophisticated analytical capability within the AI Risk Assessment Service, again without altering the mapper's REST contract:

- Isolation Forest — an unsupervised anomaly-detection technique for identifying atypical login patterns without labeled data.
- User Behavior Analytics — aggregation of longer-term behavioral signals to refine risk scoring beyond a single login event.
- Explainable AI — techniques to surface the rationale behind a given risk classification, supporting administrator trust and auditability.
- Continuous Learning — mechanisms for the scoring model to adapt over time as new labeled or semi-labeled data becomes available.

14.3 Version 4 — Potential Reinforcement Learning-Based Adaptive Policy Engine

As a longer-term possibility, a reinforcement-learning-based adaptive policy engine could allow the AI Risk Assessment Service to adjust its own risk thresholds and recommended restrictions over time based on observed outcomes. Consistent with the Concept Note's explicit direction, reinforcement learning is not part of Version 1 and is documented here strictly as a future enhancement, not as a current or near-term requirement.

14.4 Architectural Stability Across Versions

Across all versions described above, the following architectural boundaries — already established in the Concept Note and formalized throughout this SRS — remain unchanged: Keycloak retains sole responsibility for authentication and token signing; the Custom Protocol Mapper retains sole responsibility for context collection, REST communication, and claim transformation; and the AI Risk Assessment Service retains sole responsibility for risk evaluation, regardless of the sophistication of its internal scoring technique. This stability is what allows the roadmap in this section to be pursued incrementally, without revisiting the requirements defined in Sections 4 through 7 of this document.

15 Appendices

15.1 Glossary

Term	Definition
Adaptive Authorization	An authorization model in which granted privileges can vary based on the assessed context or risk of a specific access event, rather than being fixed solely by stored identity attributes.
Contextual Authentication	Authentication decision-making or post-authentication treatment that takes into account real-time signals such as network origin, device, and time of access.
Zero Trust	A security model in which no session or request is implicitly trusted based solely on prior authentication; every access is evaluated against current context and policy.
Fail-Closed	A failure posture in which, upon error or unavailability of a dependency, the system defaults to the more restrictive/conservative behavior.
Fail-Open	A failure posture in which, upon error or unavailability of a dependency, the system defaults to the less restrictive behavior (i.e., proceeds as if no risk-based adjustment applied).
Loose Coupling	An architectural property in which components interact only through well-defined, stable interfaces and can be changed independently, as required between the mapper and the AI Risk Assessment Service.

15.2 Abbreviations

Abbreviation	Meaning
FR	Functional Requirement
NFR	Non-Functional Requirement
TC	Test Case
UC	Use Case
SPI	Service Provider Interface
OIDC	OpenID Connect
JWT	JSON Web Token
REST	Representational State Transfer
TLS	Transport Layer Security

15.3 Technical References

- Keycloak Server Developer Guide — Protocol Mappers and SPI extension points.
- OAuth 2.0 Authorization Framework (RFC 6749).
- OpenID Connect Core 1.0 specification.
- JSON Web Token (JWT) standard (RFC 7519).
- IEEE 29148:2018 — Systems and software engineering — Requirements engineering.
- NIST SP 800-207 — Zero Trust Architecture.

15.4 Standards and Concepts Referenced

Standard / Concept	Relevance to this Project
OAuth 2.0	Underlying authorization framework on which Keycloak's token issuance is based.
OIDC (OpenID Connect)	Identity layer on top of OAuth 2.0 defining ID Tokens and the Protocol Mapper extension point used by this project.
JWT	Token format issued by Keycloak and modified in-memory by the Custom Protocol Mapper.
SPI (Service Provider Interface)	Keycloak's own extension mechanism, used exclusively to implement the Custom Protocol Mapper.
Protocol Mapper	The specific Keycloak SPI extension point this project implements, per AbstractOIDCProtocolMapper.
Adaptive Authorization	The overarching access-control paradigm this project demonstrates within Keycloak.
Contextual Authentication	The practice of incorporating authentication-time signals into access decisions, implemented here via the AI Risk Assessment Service.
Zero Trust	The broader security philosophy motivating per-session, context-aware evaluation as required by this SRS's design constraints.

End of Software Requirements Specification.